实验驱动学习 / 第二阶段复盘

Agent 学习讲义 02

从任务验收到真实代码实验

概念、代码、实验、错题与复习路线

学习日期：2026 年 10 月 22–25 日
项目：AgentNova / ExperimentAgent
学习环境：Windows / Anaconda / Python 3.11 / 本地 Ollama
本讲范围：11 次后续小课、7 个后续实验、真实 CPU 训练
当前关卡：G1 独立迁移待验收；已有可验收的真实训练流程

本文是本项目的阶段性学习讲义，不是 CS2680 官方讲义。
实际运行、教学模拟、独立修改与后续规划分别标记。
原始源码、笔记和报告通过附录索引保留，复盘数字以对应运行记录为准。

目录

1 本阶段究竟增加了什么 1

1.1 三种程序仍要区分 1

1.2 五类“成功”不是同一个状态 2

1.3 学习证据的记账规则 2

2 第 17 课：从目录观察中选择唯一文本文件 2

2.1 输入不是文件正文 2

2.2 第一处真实错误：在遍历中提前判断总数 3

3 第 18 课：把选择结果接到真实读取 3

3.1 固定三步流程 3

3.2 三个真实文件案例 4

4 第 19–20 课：独立任务验收器 4

4.1 为什么不能只相信最终回复 4

4.2 请求和结果必须配对 4

4.3 三个最小反例 5

4.4 验收范围仍然有限 5

5 第 21 课：受控执行 Python 脚本 5

5.1 模型提交 ID，程序决定执行方式 5

5.2 进程观察字段的含义 5

5.3 学习者实际结果 6

6 第 22 课：真实模型选择脚本与停止策略 6

6.1 任务、选择和验收分开 6

6.2 保存观察不等于模型收到观察 7

7 第 23 课：亲手组织核心循环与动作校验 7

7.1 本题教学协议 7

7.2 真实提交中的错误及修复 7

7.3 三层动作检查 8

7.4 退出位置与预算 8

8 第 24 课：指标函数的输入契约 9

8.1 数据合法与数值达标分开 9

8.2 两个短路判断 9

9 第 25 课：真实指标文件的读取、解析与验收 9

9.1 三步接口转换 9

9.2 五类实际文件结果 10

9.3 学习者对异常返回的解释 10

10 第 26 课：关联本次运行，防止旧结果冒充 10

10.1 共享文件名带来的错误 10

10.2 目录边界与文件内标记同时检查 11

10.3 五例结果与一次因果纠正 11

10.4 这项关联保证的边界 11

11 第 27 课：用真实线性回归替换教学常量 12

11.1 固定实验设置 12

11.2 指标如何真正产生 12

11.3 学习者真实结果 12

11.4 保存产物和独立复算 13

12 从学习问题提炼出的易错点 13

13 运行与测试证据总账 14

14 复现命令与阅读源码的顺序 15

14.1 统一环境 15

14.2 不调用模型的复习路线 15

14.3 真实模型复习路线 15

15 自测题与对照答案 16

15.1 自测题 16

15.2 对照答案 17

16 下一阶段与恢复学习入口 17

17 附录 A. 第 17–27 课完整任务与原始教学细节 18

17.1 第 17 次动手：用目录观察判断能否继续 18

17.1.1 核心概念 18

17.1.2 你需要实现的函数 18

17.1.3 返回契约 19

17.1.4 运行验收 19

17.1.5 交给我检查 20

17.1.6 首次提交检查 20

17.1.7 修正后复验 20

17.2 第 18 次动手：将选择函数接到真实读取步骤 20

17.2.1 本次只学一个连接点 20

17.2.2 先看核心代码 21

17.2.3 为什么还要检查读取结果 21

17.2.4 运行三个真实案例 22

17.2.5 与 l14 真实 Agent 对照 23

17.2.6 本次理解检查 23

17.2.7 学习者运行验收 23

17.3 第 19 次动手：根据执行证据验收 Agent 23

17.3.1 本次核心区别 23

17.3.2 看什么证据 24

17.3.3 先看两个决定性的分支 24

17.3.4 三个稳定案例 25

17.3.5 这版验收的边界 26

17.3.6 理解检查 26

17.3.7 学习者验收记录 26

17.4 第 20 次动手：真实模型入口同时显示循环与验收状态 26

17.4.1 连接位置 26

17.4.2 运行 . . . 27
17.4.3 报告里的两个结果 . . . 27
17.4.4 验收器适用范围 . . . 28
17.4.5 连接验收测试 . . . 28
17.4.6 理解检查 . . . 29
17.4.7 学习者验收记录 . . . 29
17.5 第 21 次动手：受控执行 Python 脚本 . . . 29
17.5.1 本课目标 . . . 29
17.5.2 工具只能接收 script_id . . . 29
17.5.3 看核心执行代码 . . . 30
17.5.4 四个字段，不能混为一谈 . . . 31
17.6 子进程里的 RuntimeError 不会直接跳进父工具的 except RuntimeError . . . 31
17.7 超时另走异常分支 . . . 31
17.7.1 运行三个真实案例 . . . 31
17.7.2 当前控制范围 . . . 32
17.7.3 本次理解检查 . . . 33
17.7.4 学习者验收记录 . . . 33
17.8 第 22 次动手：真实模型选择脚本，策略控制继续或停止 . . . 33
17.8.1 本次分工 . . . 33
17.8.2 新入口，不改变原只读入口 . . . 33
17.8.3 三个真实模型任务 . . . 34
17.8.4 几种限制分别控制什么 . . . 34
17.8.5 怎样读输出 . . . 35
17.8.6 连接测试 . . . 36
17.8.7 理解检查 . . . 36
17.8.8 学习者验收记录 . . . 36
17.9 第 23 次动手：自己组织 Agent 的核心循环 . . . 37
17.9.1 本次写哪里 . . . 37
17.9.2 输入和消息契约 . . . 37
17.9.3 返回值契约 . . . 38

17.9.4 八项验收 . . . 39
17.9.5 第一小步 . . . 39
17.10 第 24 次动手：进程成功与实验指标验收 . . . 39
17.10.1 新知识点 . . . 40
17.10.2 小练习：verify_metrics . . . 40
17.10.3 第一小步 . . . 41
17.11 第 25 次动手：读取真实指标文件并验收 . . . 41
17.11.1 数据流 . . . 42
17.11.2 第一步：读文件 . . . 42
17.11.3 第二步：把字符串解析成数据 . . . 42
17.11.4 第三步：调用学习者的验收函数 . . . 43
17.11.5 在 Anaconda 中运行 . . . 43
17.11.6 本次理解检查 . . . 44
17.12 第 26 次动手：执行脚本，只验收本次运行的指标 . . . 44
17.12.1 为什么只读 metrics.json 不够 . . . 44
17.12.2 一、父进程分配运行上下文 . . . 44
17.12.3 二、脚本在哪写文件 . . . 44
17.12.4 三、执行失败时不读取结果 . . . 45
17.12.5 四、只读本次目录，并检查编号 . . . 45
17.12.6 五个真实运行案例 . . . 45
17.12.7 本次理解检查 . . . 46
17.13 第 27 次动手：真实线性回归训练与指标验收 . . . 46
17.13.1 这次变化 . . . 46
17.13.2 实验规则预先固定 . . . 47
17.13.3 训练脚本负责什么 . . . 47
17.13.4 运行器与验收器负责什么 . . . 47
17.13.5 本次目录保存哪些产物 . . . 48
17.13.6 实际测量（AI 首次运行） . . . 48
17.13.7 在 Anaconda 中运行 . . . 48
17.13.8 本次理解检查 . . . 49

18 附录 B. 当前核心源码全文 **49**

18.1 exercises/select_text_file.py 49

18.2 labs/l15_directory_read_workflow.py 50

18.3 experiment_agent/verification.py 52

18.4 experiment_agent/tools/script_tools.py 55

18.5 experiment_agent/tool_policy.py 57

18.6 labs/l18_real_script_agent.py 58

18.7 exercises/g1_agent_loop.py 62

18.8 exercises/verify_metrics.py 64

18.9 experiment_agent/metrics_workflow.py 65

18.10 experiment_agent/experiment_workflow.py 66

18.11 experiments/linear_regression/train.py 70

18.12 labs/l21_real_linear_regression.py 72

19 附录 C. 固定学习资料、证据和训练产物索引 **74**

19.1 docs 75

19.2 exercises 75

19.3 experiment_agent 75

19.4 experiments 76

19.5 labs 76

19.6 learner_evidence 77

19.7 lessons 77

19.8 notes 78

19.9 reports 79

19.10 selected_runs 80

19.11 templates 81

19.12 tests 81

19.13 项目入口与环境 81

20 附录 D. 学习者对话与终端证据摘录 **82**

20.1 对话摘录的来源说明 82

20.2 目录选择、读取和证据 82
20.3 脚本与消息 82
20.4 核心循环的实现过程 82
20.5 指标与 JSON 83
20.6 本次产物与真实训练 83

学习区间：2026-10-22 至 2026-10-25。归档日期：2026-10-25。项目：AgentNova / ExperimentAgent。环境：Windows、Anaconda、Python 3.11、本地 Ollama。

本讲接续 Lecture 01，覆盖第 17–27 次小课和 l15–l21；为了让后半阶段的能力可以独立复习，也回顾第一讲的工具、消息、策略和循环接口。两讲合起来覆盖目前第 1–27 次小课。这里的课号是本项目的教学小节编号，不是 CS2680 官方授课编号。

公开版说明：Windows 用户目录用 USER 占位，私人附件的内部位置不公开；`run_id` 是教学程序生成的运行标记，保留以便关联报告。源码、完整逐课任务、测试与报告在固定快照中，manifest.json 的哈希对应脱敏后的实际字节。旧版本的待办和失败记录保留为历史，不应误读为最终状态。

1 本阶段究竟增加了什么

第一讲跑通了模型提出工具请求、Python 执行、结果回传与最终回答。本阶段把“能运行”推进到“能用证据判断任务是否完成”，最后执行了真实的 CPU 线性回归训练。

```
用户目标和可信验收条件
  → 模型决策或固定 Python workflow
  → 工具请求结构与参数检查
  → 真实目录、文件或子进程观察
  → 运行编号和产物路径关联
  → 独立指标规则验收
  → 保留通过、未达标、执行失败各类结果
```

1.1 三种程序仍要区分

形式	下一步的决定者	本阶段例子
固定 workflow	Python 预设控制流	l15、l19、l20、l21
模型驱动循环	模型结合观察提出动作，Python 执行和约束	l14 discover、l18
测试模型替身	预先写好的可预测动作	l16、G1 练习及多数单元测试

真实训练不自动意味着训练配置已由 Agent 选择。l21 确实更新了模型参数和计算验证 MSE，但两个配置仍由可信 Python 程序指定；模型配置决策是后续任务。

1.2 五类“成功”不是同一个状态

层次	状态例子	它实际说明什么
工具调用	success	单次工具操作成功
模型循环	completed	按协议收到最终回答并结束
子进程	`exit_code`=0、未超时	进程正常退出
产物来源	`run_id_matched`	文件标记与本次预期编号一致
任务验收	passed	已记录证据满足本题验收条件

例如进程退出码为 0，但没有生成文件，文件任务仍失败。又如循环 completed，但模型没有实际读取目标文件，读取任务仍失败。报告应同时展示执行情况和验收结果。

1.3 学习证据的记账规则

“学习者独立运行”“学习者在提示后实现”“AI 实现后运行通过”“学习者能独立解释”分别记录。不能把全量测试由 AI 跑过，改写成学习者全部亲自运行；不能把按提示拼接并由 AI 修复的完整循环，记成独立从零实现。

截至本次归档，AI 全量测试有 122 项产品测试和 28 项练习通过的记录；学习者最近分别提交 l19 的 8 项、l20 的 12 项、l21 的 7 项测试通过结果。它们是不同证据范围。

2 第 17 课：从目录观察中选择唯一文本文件

2.1 输入不是文件正文

`list_directory` 返回名称、类型、相对路径和 truncated。选择函数只处理这些观察数据，不再访问文件系统，也不调用模型。它的任务是判断“是否有且只有一个普通.txt 文件”。

按以下顺序判断很重要：先保留工具失败，再检查观察是否完整，最后才统计候选。truncated=true 时看到一个匹配项，只能说明当前返回部分中有一个，不能证明整个目录唯一。

输入情形	选择结果
目录工具失败	保留原失败
结果截断	`incomplete_directory_listing`
完整观察，无候选	`no_text_file`
完整观察，多个候选	`multiple_text_files`

输入情形	选择结果
完整观察，唯一普通文本文件	返回观察中已有的 `relative_path`

候选要求 type=file，扩展名大小写不敏感；目录和符号链接不算本题的普通文件。合法路径中的空格不能擅自去掉，也不能根据文件名猜一个路径替代 observed `relative_path`。

2.2 第一处真实错误：在遍历中提前判断总数

学习者依据提示补全函数，首次 8 项中 5 项通过、3 项失败。数量判断与最后 return 仍缩进在 for 内，产生三类问题：空目录没有进入循环而返回 None；首项非文本时过早判定没有候选；多个文件时看到第一项就过早成功。

```
正确层次：
先处理不允许继续的观察
for 每个条目：
    满足条件时加入 candidates
遍历全部完成后：
    判断候选总数，返回唯一结果或失败
```

学习者自行把数量判断移到循环外，复验 8 项全部通过。此处确认的是题目契约，不是任意畸形输入的通用防护，也不是已经完成真实模型集成。

3 第 18 课：把选择结果接到真实读取

3.1 固定三步流程

l15 的执行顺序固定为 `list_directory` → `select_unique_text_file` → `read_text_file`。选择失败后 return，后面没有读取；选择成功后，才把选择结果中的 `relative_path` 交给原分发器。两种工具各自保持已有权限检查。

```
selection = select_unique_text_file(directory_result)
if selection["status"] != "success":
    return {"status": "failed", "result": selection, "trace": trace}
read_result = call_tool(tools, "read_text_file", {
    "relative_path": selection["relative_path"],
})
```

最后的外层状态来自 `read_result`，而不是 selection。选中了正确文件，文件也可能不是 UTF-8 或没有读取权限。

3.2 三个真实文件案例

案例	停止位置或结果	学习重点
唯一文本文件	实际读取两行中文	路径来自目录观察
多个文本文件	`multiple_text_files`	选择失败就不读取
唯一但非法 UTF-8	`invalid_encoding`	选择成功不等于读取成功

非法编码实际由底层读取抛出 UnicodeDecodeError，`call_tool` 将其转换为结构化错误。except 转换错误，后续流程决定退出；不要把这两个职责合并成一句“except 自动停止 Agent”。

学习者独立运行三例，并正确解释提前 return 和最终状态传播。新增连接测试 6 项，本阶段产品测试从 50 项增加到 56 项。

4 第 19–20 课：独立任务验收器

4.1 为什么不能只相信最终回复

模型可能说“已经读取”，却只列过目录；也可能真实读取了另一个文件。循环 completed 只说明协议终止正常，不能证明任务正确。验收器需要看可信运行时保存的请求和工具结果。

`verify_unique_text_read` 不重新执行工具。它从原生消息轨迹配对 assistant.`tool_calls` 和 role=tool 结果，确认任务目录的完整观察、唯一候选、后续实际读取路径及读取成功的内容证据。

4.2 请求和结果必须配对

孤立的工具结果、缺失的结果、结果名称与待执行请求不一致、下一次模型消息出现时仍有未完成请求，都不能作为完整执行证据。现有运行器串行执行，因此验收器按顺序配对。这里没有实现任意并发轨迹的通用配对，也没有对日志做防篡改签名。

目录观察必须早于读取；读取目标必须等于目录观察选出的路径。目录被截断时，验收器不能凭部分结果宣布唯一。内容必须来自成功读取结果，空文本本身也可以是有效读取。

4.3 三个最小反例

案例	循环状态	验收结果
查看目录后读取目标	completed	passed / `verified_read`
只列目录后声称已读取	completed	failed / `missing_read_evidence`
读了 other/other.txt	completed	failed / `wrong_file_read`

第三例期望 data/hello.txt，实际 other/other.txt。学习者正确根据 `expected_path` / `actual_path` 指出问题，而不是根据模型口头“完成”判断。

第 20 课把验收接进真实 l14 discover。学习者观测三次模型请求，`list_directory` 后 `read_text_file`，最终 completed + passed / `verified_read`，约 4.993 秒。AI 较早运行约 15.592 秒，二者条件未做受控比较，不据此宣称优化。

4.4 验收范围仍然有限

当前读取验收主要核对实际读取行为，没有全面评价最终回答语义；保守地拒绝不完整或错误的读取证据，也没有为所有重试恢复路径设计通用评分。它是本题可检验的任务规则，不是所有 Agent 的统一裁判。

5 第 21 课：受控执行 Python 脚本

5.1 模型提交 ID，程序决定执行方式

`run_python_script` 只接受固定 normal、error、timeout 三种 `script_id`。项目根目录、解释器、时限和命令形式由可信 Python 决定。原默认注册表仍是只读；执行能力必须显式启用，工具参数不能覆盖 `project_root`、command、args 或 `python_path`。

执行使用当前 sys.executable，shell=False、stdin=DEVNULL、UTF-8 输出捕获和超时。-I 降低 Python 环境注入影响，但不等于文件、网络和操作系统权限都被隔离。

5.2 进程观察字段的含义

字段	正确解释
stdout	普通输出，可以包含结果和进度
stderr	诊断输出，不必然表示失败

字段	正确解释
`exit_code`	正常返回的实际退出码；0 表示正常退出
`timed_out`	是否触发了执行时限
`duration_seconds`	工具执行观测耗时，不等于模型整轮耗时

normal 的 stderr 有一条“计算已完成”的诊断，但 `exit_code`=0 且未超时，因此工具 success。error 抛 RuntimeError，退出码 1，归类 `process_failed`。timeout 在约 2 秒后停止，保留已捕获输出，`exit_code`=null，不伪造一个正常退出码。

5.3 学习者实际结果

学习者三例耗时约 0.021、0.020、2.012 秒，指定 12 项测试通过，2.138 秒。学习者明确解释“stderr 可以有诊断信息，不能只看是否为空”。测试还覆盖拒绝非法参数、越界名单、非文件、启动错误和超时输出解码。

这是可信脚本的受控执行接口，仍继承当前用户可用权限。任意不可信代码、进程树管理、输出和资源配额、网络隔离不在已完成范围中。

6 第 22 课：真实模型选择脚本与停止策略

6.1 任务、选择和验收分开

l18 将三个任务描述及单个执行工具交给真实本地模型，由模型选择 `script_id`。`EXPECTED_SCRIPT` 只在运行后核对，没有替模型填参数。每任务最多实际调用一次执行函数，最多四次模型请求；参数修正预算沿用已有规则。

学习者真实观察：normal 两次模型请求，执行成功后把结果送回模型总结；error 和 timeout 各一次请求，执行后由策略停止。每例执行函数调用一次，三例 `expected_behavior_matched` 都为 True。

案例	工具结果	策略 / 循环	学习者整轮耗时
normal	success / 0	continue 后 completed	47.498 秒
error	`process_failed` / 1	stop 后 failed	0.361 秒
timeout	`tool_timeout` / null	stop 后 failed	2.348 秒

这些耗时包含模型请求和可能的加载，不能与子进程约几十毫秒的时间混用。学习者指定

8 项连接测试通过，2.211 秒。

6.2 保存观察不等于模型收到观察

```
执行失败
  → append 失败工具结果到 messages
  → 策略返回 stop
  → Python 返回失败
  → 没有下一次 chat(messages, tools)
```

学习者正确指出：模型尚未看到失败结果。终端停止原因由 Python 根据真实观察打印，不是模型在看到错误后作出的总结。`expected_behavior_matched`=True 表示演示符合预期，不表示异常脚本成功。

7 第 23 课：亲手组织核心循环与动作校验

7.1 本题教学协议

G1 练习使用简化动作字典。model(messages, `tool_descriptions`) 返回 `tool_call` 或 final；assistant.content 保存动作字典，tool.content 保存结果字典。它不直接等同 Ollama HTTP 原生消息协议。

```
{"type": "tool_call", "name": "read_text_file",
 "arguments": {"relative_path": "data/notes.txt"}}
{"type": "final", "content": " 最终回答"}
```

messages 和 `corrections_used` 在循环外初始化。每轮先请求模型并保存动作，再校验、执行；工具观察先保存，再按策略 continue、correct 或 stop。steps 数的是模型决策次数，不是消息条数或文件读取次数。

7.2 真实提交中的错误及修复

第一次检查磁盘仍为骨架，NotImplementedError 导致测试无法验证实现。再次保存后看到尝试版本：工具执行脱离函数或循环，continue 不在循环内，模块因此 SyntaxError，八项行为测试尚未实际执行。

另外三个判断表达式没有用 and 组成 if，后面却无条件返回 completed。学习者随后明确请求 AI 修复。AI 修复控制流并补齐校验，8 项测试通过。后续还观察到同一路径版本改

变与编辑器旧内容不一致，重新核对实际导入路径、磁盘内容和测试前后源码哈希；没有证据能确定是哪个编辑器或同步环节造成覆盖。

这一过程的重要习惯是：核对文件绝对路径、已保存内容和测试实际导入来源，不根据“编辑器看起来改了”推断磁盘已更新。源码改变后，旧的测试通过结论只属于当时版本。

7.3 三层动作检查

先确认 action 是字典，才能调用 get。有效 final 要有非空白字符串正文。有效工具请求要有正确 type、非空白字符串 name 和 arguments 字段。

```
if (
    action.get("type") == "final"
    and isinstance(action.get("content"), str)
    and action["content"].strip()
):
    # 返回 completed，保留原正文。
    ...
```

and 从左到右短路，类型不对就不会调用 strip；strip 在这里检查内容，不改写保存的原值。未知动作、非字典动作、缺请求字段都返回 `invalid_model_response`，不执行工具。

情形	负责处理的层
缺少 arguments 字段	循环判请求结构非法并停止
有 arguments={}，但缺 `relative_path`	分发器判 `invalid_arguments`，策略可允许一次修正
已注册工具真实执行抛已知异常	分发器转换错误，再交策略

7.4 退出位置与预算

return 结束函数；continue 进入下一轮。修正次数跨整个任务累计，工具成功不重置。`max_steps` 的超限返回在 for 外、函数内：最后一次工具成功但没有剩余模型请求生成 final，仍可能 `max_steps_exceeded`。

本题最终 8 项行为测试通过，但完整循环经过提示、学习者尝试和 AI 修复，不能记为独立无提示实现。G1 的独立迁移检查仍待完成。

8 第 24 课：指标函数的输入契约

8.1 数据合法与数值达标分开

`verify_metrics` 接收已经解析的数据，不读文件、不执行实验。mse 必须是 int/float 中的有限非负数，显式排除 bool，且不自动把数值字符串转换成数字。0 和恰好等于阈值的值允许通过。

输入	返回分类
非字典或缺 mse	`invalid_metrics`
mse 为字符串、None 或 bool	`invalid_metrics`
mse 为 NaN、无穷或负数	`invalid_metrics`
合法 mse 大于 `max_mse`	`mse_above_threshold`
合法 mse 不大于 `max_mse`	`verified_metrics`

阈值由可信调用方提供，不从模型回复或指标文件里取一个更宽松的值。本练习针对常规范围的 Python int/float，没有完整实现任意大小输入的资源限制。

8.2 两个短路判断

```
if not isinstance(metrics, dict) or "mse" not in metrics:
    return {"status": "failed", "reason": "invalid_metrics"}
mse = metrics["mse"]
```

or 左边已真就不执行右边。因此对 None 不会继续检查字典字段。取出值后，先检查类型，再调用 math.isfinite，最后比较是否非负和是否超过阈值。

bool 是 int 的子类，isinstance(True, int) 为真，不能让 True 被当作合法实验 MSE。NaN 的普通大小比较也不能替代有限性检查。

学习者正确拒绝字符串”0.04”，在前两个判断提示后提交完整实现；AI 保留其代码，指定 8 项测试通过。这里的失败分成 “数据不合法” 和 “数据合法但任务未达标”，后续解释错误时需要保留这种区别。

9 第 25 课：真实指标文件的读取、解析与验收

9.1 三步接口转换

```
相对路径
  → read_text_file: 返回 UTF-8 正文字符串
  → json.loads: 字符串转 Python 数据
  → verify_metrics: 数据结构、数值与阈值规则
```

正文已经由受控文件工具读出，因此使用 json.loads；json.load 则接收已打开的文件对象。可解析 JSON 不一定是字典，null、数组或数字仍需后面的指标结构检查。

9.2 五类实际文件结果

文件情况	最终分类	执行到哪一步
{”mse”: 0.04}	`verified_metrics`	全部三步
不存在	`file_not_found`	读取
{”mse”: 0.04,}	`invalid_json`	解析
{”mse”: ”0.04”}	`invalid_metrics`	指标验收
{”mse”: 1.2}	`mse_above_threshold`	指标验收

9.3 学习者对异常返回的解释

学习者准确复述：读取成功只代表拿到正文；json.loads 抛 JSONDecodeError，except 保存解析错误，分支中的 return 结束函数，后续 `verify_metrics` 不执行。捕获异常的是 except，实际结束函数的是 return。

学习者五例预期行为全部匹配，指定 8 项测试通过，0.028 秒。该次提示符是 base，未提供解释器路径，因此不能记成已确认 agent-study。解析器错误文案及列号与 AI 环境略有差别，但都被正确分类为 `invalid_json`，不为文案差别重跑已通过案例。

工具和解析使用 success，验收使用 passed。外层最终状态取自最后真正执行的阶段，不能因为第一步成功就掩盖后续失败。此时只验证指定文件当前内容，还没有证明是本次运行生成。

10 第 26 课：关联本次运行，防止旧结果冒充

10.1 共享文件名带来的错误

如果上次已经生成 metrics.json，本次脚本却没有写文件，直接读取共享位置可能把旧数值误认为新数值。仅看 stdout、文件存在或修改时间，不足以完成本题的来源关联。

本程序由父进程生成 uuid4().hex，创建 runs/<run_id>/，拒绝复用已存在目录。子进程 cwd 设为新目录，相对写入 metrics.json；程序预期读取位置始终由可信上下文确定，不采用模型或 stdout 建议的替代路径。

10.2 目录边界与文件内标记同时检查

```
新运行编号和目录
  → 固定可信脚本执行
  → 进程成功后，把读取工具根目录缩小为本次目录
  → 读取固定 metrics.json
  → 解析后 check_run_id
  → 验收 mse
```

`run_metrics_file_workflow` 的 `expected_run_id` 是可选参数。不给它时保持 l19 三步；给它时在解析后加入编号检查。编号缺失或不一致直接 `run_id_mismatch`，mse 再小也不继续放行。

10.3 五例结果与一次因果纠正

案例	进程观察	最终原因
normal	正常写入本次结果	`verified_metrics`
`missing_metrics`	退出码 0，旧结果仍在，本次不写	`file_not_found`
`wrong_run_id`	退出码 0，数值合法但编号错误	`run_id_mismatch`
error	已写文件后抛异常	`process_failed`，不读取
timeout	已写文件后等待超时	`tool_timeout`，不读取

学习者最初问“是不是因为 Workflow status=failed，所以找不到指标”。这里因果顺序相反：绑定本次目录，实际没有生成目标，读取返回 `file_not_found`，外层再汇总为 failed。旧文件属于另一运行目录，不能替代本次输出。学习者随后正确复述这一因果关系。

学习者在 agent-study 实际运行五例，指定 12 项测试通过，0.842 秒。测试还覆盖两个运行互不覆盖、编号碰撞拒绝创建、固定脚本越界、非法 case、启动失败以及可信时限检查。

10.4 这项关联保证的边界

关联成立的前提是固定可信脚本、运行目录没有被外部并发篡改。`run_id` 不是防伪签名，不证明发生了训练，也不能阻止恶意脚本主动伪造标记。当前控制不等于任意代码沙箱。

11 第 27 课：用真实线性回归替换教学常量

11.1 固定实验设置

数据为 y=3x+2+ 噪声，x 在 [-1,1] 均匀采样，噪声标准差 0.1。训练 128 个样本、验证 64 个样本，使用两个独立固定种子。两组从 weight=0、bias=0 开始，进行相同 100 步全批量梯度下降，只改变学习率 0.1 与 0.01。

验证集不参与梯度、早停或步数选择。这里是小型合成验证实验，尚未建立最终保留测试集，不用它宣称真实任务的普遍泛化能力。

11.2 指标如何真正产生

```
prediction = weight * x + bias
error = prediction - y
MSE = sum(error ** 2) / 样本数
gradient_weight = 2 * mean(error * x)
gradient_bias = 2 * mean(error)
weight -= learning_rate * gradient_weight
bias -= learning_rate * gradient_bias
```

`fit_line` 只接收训练数据。更新结束后分别计算训练和验证 MSE，metrics.json 顶层 mse 等于 `validation_mse`，兼容原来的验收函数。

记录初始误差用于确认确实改善；均值基线用训练标签均值作为常数预测，再在验证集评估。基线不能使用验证标签均值拟合，否则比较的训练信息不一致。

11.3 学习者真实结果

学习率	步数	训练 MSE	验证 MSE	任务结果
0.1	100	0.01132177	0.01207715	passed
0.01	100	0.89226171	0.87744596	failed / `mse_above_threshold`

两组初始训练 MSE 都是 6.77084887，初始验证 MSE 都是 5.57655214，均值基线验证 MSE 都是 3.19075572。`same_conditions`=True 表示数据指纹和除学习率外的配置一致。

学习率 0.1 的参数约为 weight=2.99633485、bias=2.00067889；学习率 0.01 约为 weight=1.40792217、bias=1.71814113。学习者训练循环耗时约 0.001462 和 0.001470 秒，只记录，不作微小性能排名。

第二组进程正常退出、编号匹配，但 0.87744596 大于预设阈值 0.1。因此是指标未达标，而不是 Python 执行失败。它的验证误差已经显著下降，不能说“没有学到任何东西”；在这个固定预算下不足以满足目标。也不能推断所有问题都应该使用更大学习率。

11.4 保存产物和独立复算

文件	用途
config.json	学习率、步数、样本数、种子、初始化
data.json	实际训练和验证数据
model.json	学到的 weight 和 bias
`validation_predictions`.json	x、真实标签、预测值
metrics.json	本次编号、各项指标、配置、数据指纹、脚本哈希、Python 版本

新增测试从保存的数据和模型重新计算验证预测及 MSE，不调用训练脚本的 mse 函数，并核对预测文件和报告；还用手算误差及已知无噪声直线检查数学行为。相同配置重跑时，编号不同，数据、参数和误差可复现，耗时允许不同。

学习者独立运行两组真实训练，指定 7 项测试通过，0.528 秒，并正确区分了进程成功、编号匹配与指标未达标。运行时通用验收器本身仍只检查结构、数值和阈值，独立产物复算目前在测试中，不能扩写成通用防伪评估已经完成。

12 从学习问题提炼出的易错点

原问题或误解	修正后的判断依据
在遍历第一项后就判断总候选数	先收集完，再统计
选择成功等于读取成功	最终看真实读取结果
completed 等于任务完成	独立检查任务证据
stderr 非空必然失败	看退出码、超时和工具状态
append 后模型已经知道结果	还需下一次模型调用

原问题或误解	修正后的判断依据
except 本身决定停止	except 转换错误，策略或 return 控制退出
三行判断表达式自动构成条件	必须用 if 和逻辑连接词
continue 可以放在函数任意位置	只能位于循环内部
有旧文件就可以补救本次缺失	只读本次目录，禁止回退
workflow failed 导致文件不存在	文件缺失导致读取失败，再汇总失败
`exit_code`=0 保证指标达标	执行层和任务验收层分开
学习率 0.01 的结果失败代表程序崩溃	本例是固定预算下验证 MSE 未达标

学习时需要保留失败作为可复盘证据，不删除测试、不放宽阈值、不替换运行结果来制造通过。预期故障被正确拒绝，也可以是测试成功。

13 运行与测试证据总账

阶段	学习者确认的运行或测试	实现归属与待补
第 17 课	AI 复验筛选题 8 项通过	有提示实现，学习者自行修缩进
l15	三例实际文件输出	AI 连接流程，学习者解释返回与状态传播
l16	三例、指定 13 项，0.011 秒	AI 验收器，学习者理解路径证据
l14 discover 接验收	completed + passed，4.993 秒	AI 接入，学习者独立运行
l17	三例、指定 12 项，2.138 秒	AI 执行工具，学习者理解 stderr
l18	三例真实模型、指定 8 项，2.211 秒	AI 实现，学习者理解失败未回传
G1	AI 修复与补齐校验后 8 项通过	不能记作无提示独立完整实现
指标函数	AI 检查学习者代码，8 项通过	提示后完成，AI 未改写提交
l19	五例、指定 8 项，0.028 秒	学习者准确解释解析失败后 return
l20	五例、指定 12 项，0.842 秒	学习者纠正失败因果解释
l21	两组真实训练、指定 7 项，0.528 秒	学习者准确解释指标阈值失败

产品测试随新增功能从第一讲的 50 项，依次增长到 56、69、75、87、95、103、115、122 项。练习为超时映射 4 项、目录筛选 8 项、核心循环 8 项、指标函数 8 项，共 28 项。不同阶段的“Ran N tests”是当时范围，不应把历史数字当成同一次完整运行。

14 复现命令与阅读源码的顺序

14.1 统一环境

```
cd /d E:\study\agent
conda activate agent-study
python -c "import sys; print(sys.executable)"
python -X utf8 -m unittest discover -s tests -v
python -X utf8 -m unittest discover -s exercises -v
```

本阶段脚本代码只需标准库；PDF 和 LaTeX 制作环境属于文档工具，不是训练或 Agent 的依赖。不要将文档运行时路径或私人附件位置复制进公开实验配置。

14.2 不调用模型的复习路线

```
python -X utf8 -m labs.l15_directory_read_workflow
python -X utf8 -m labs.l16_agent_task_verifier
python -X utf8 -m labs.l17_controlled_python_script
python -X utf8 -m labs.l19_metrics_file_workflow
python -X utf8 -m labs.l20_run_scoped_metrics
python -X utf8 -m labs.l21_real_linear_regression
```

l20 和 l21 会创建新的运行目录；旧目录不删除、不复用。重新运行会覆盖相应汇总报告，因此本讲固定快照用于保留归档时的证据。

14.3 真实模型复习路线

```
python -X utf8 -m labs.l14_multi_tool_agent --case discover
python -X utf8 -m labs.l18_real_script_agent --case all
```

这两条需要本机 Ollama 与模型，会发生真实推理。归档和重新排版不需要再运行它们；也不要把替身测试通过当作重新完成了真实模型实验。

阅读源码建议顺序：`select_text_file` → l15 → verification → `script_tools` → l18 → `g1_agent_loop` → `verify_metrics` → `metrics_workflow` → `experiment_workflow` → train.py。附录保留完整版本，不能把本文局部代码片段不加上下文直接拼成函数。

15 自测题与对照答案

先遮住答案独立回答，再定位相应真实分支和测试。答对解释与能独立修改实现分别记录。

15.1 自测题

1. truncated=true，但只有一个可见.txt，为什么不能证明唯一？
2. 把 candidates 数量判断放在 for 内，会破坏哪三种情况？
3. 模型说已读取而只列了目录，验收器需要找什么缺失证据？
4. normal 有 stderr，为什么仍可能 success？
5. 失败观察已 append 后策略 stop，模型是否已经知道错误？
6. action=None 时先用 get 会怎样？如何在访问字段前避免？
7. 缺 arguments 与 arguments={} 为什么属于不同层级？
8. 工具最后一次读取成功，为何仍可能 `max_steps_exceeded`？
9. bool 为什么不能只靠 int/float 判断排除？
10. JSON 语法正确是否保证 mse 合法？给出两个反例。
11. 旧文件存在，本次文件缺失，外层 failed 的原因链是什么？
12. 文件编号错但 mse 很小，是否应继续验收数值？
13. 脚本写完文件后抛异常，为什么不直接读取该文件通过任务？
14. 训练 mse 和验证 mse 哪一个用于当前阈值？验证集是否参与梯度？
15. 两个学习率对照必须固定哪些条件？
16. 运行编号匹配能否证明数值确实经过训练，或阻止恶意脚本？
17. 训练循环耗时和模型整轮耗时为什么不能直接比较？
18. 当前已通过真实训练，为何仍不能宣布完整 Agent 或 G1/G2 全部验收？

15.2 对照答案

1. 观察不完整，未返回部分仍可能有候选；应停止或另设计完整获取流程。
2. 空目录返回 None；首项无关时过早判零；首项匹配时漏掉后续多个候选。
3. 匹配请求的成功 `read_text_file` 工具观察，以及正确目标路径与顺序。
4. stderr 是诊断通道；正常退出且未超时仍可成功。
5. 没有下一次模型请求就尚未发送；Python 保存日志与模型接收不是同一动作。
6. 会产生 AttributeError；先 isinstance(action, dict)，不合法返回结构化诊断。
7. 缺字段是动作结构不完整；字段存在但工具所需内容缺失，由分发器处理参数错误。
8. 步数按模型决策计；还需要一次 final，而预算已用尽。
9. bool 是 int 子类，必须显式排除。
10. 例如 mse="0.04" 或顶层 []，能解析但不满足指标契约。
11. 只读本次目录 → 本次目标不存在 → `file_not_found` → 外层 failed。
12. 不应；先拒绝来源不符，不能用好数值替代证据。
13. 当前任务要求执行成功且产物合格；部分结果不能掩盖失败执行。
14. 验证 MSE；验证数据不参与梯度更新。
15. 数据和种子、划分、初始化、训练步数、评价规则、运行条件；保留失败与成本。
16. 不能；编号是可信流程中的关联标记，不是签名或完整训练真实性证明。
17. 计时范围不同，整轮可能包含模型加载、提示计算和工具开销。
18. 训练仍是固定 workflow，完整核心循环独立实现/迁移、模型配置选择和系统课程任务还未全部验收。

16 下一阶段与恢复学习入口

已经具备：真实模型工具选择、独立读取验收、受控脚本、结构化错误与停止预算、本次产物关联、真实 CPU 训练与可复算数据。下一步可把受控训练配置选择交给模型，再用同一套独立验收判断结果；随后设计有限恢复策略，不能遇到未达标就无上限地改参数。

仍待完成的重点包括 G1 独立迁移、完整实验配置与运行信息的独立实现、课程讲义/论文的系统阅读、保留测试集评估、恢复与持久化，以及更完整权限隔离。RAG、MCP、多 Agent 和推理服务优化不因本讲开始训练而被自动记为掌握。

推荐恢复顺序：先独立复述进程/来源/指标三层，再从本讲自测中选一处控制分支做修改；只有解释和实现证据齐全，才推进相应关卡。讲义归档不会安排自动学习或新增后台训练。

17 附录 A. 第 17–27 课完整任务与原始教学细节

以下保留归档时逐课全文，包括最初待办、失败过程及后续验收。正文已汇总最终状态；历史文字不应单独当成当前状态。

17.1 第 17 次动手：用目录观察判断能否继续

学习日期：2026-10-22。预计 15–20 分钟。前置：已独立完成 l14 三个真实任务，能区分目录元数据与文件正文。

本课由学习者亲自实现一个小函数。AI 只准备函数骨架和 8 项行为验收；不预先填写答案，也不调用本地模型。

17.1.1 核心概念

工具返回 success 说明这次工具操作成功，不等于已满足用户目标。例如 `list_directory` 成功返回一部分条目时，truncated=true；只看到一个.txt，不能证明整个目录只有一个.txt。不能把“暂时没看到”当作“不存在”。

本题任务要求是“目录中有且只有一个普通.txt 文件时，使用其路径继续读取”。因此先确认观察成功且完整，再判断候选数。本题遇到不完整结果就返回失败；分页、继续搜索等恢复方式留到后续。

17.1.2 你需要实现的函数

文件：exercises/`select_text_file.py`，函数：`select_unique_text_file(directory_result)`。

它接收已经获得的目录结果，不调用 `list_directory`、不读取正文、不请求模型；本题先练习确定性的数据处理。暂不注册成第三个工具，也不改变真实模型循环或全局停止策略。

本题假设输入符合现有目录工具的结构；暂不增加任意畸形输入校验。

输入例子：

```
{
  "status": "success",
  "entries": [
    {"name": "hello.txt", "type": "file", "relative_path":
    ↪  "labs/fixtures/hello.txt"},
    {"name": "README.md", "type": "file", "relative_path":
    ↪  "labs/fixtures/README.md"}
  ],
```

```
  "truncated": false
}
```

期望输出：

```
{"status": "success", "relative_path": "labs/fixtures/hello.txt"}
```

17.1.3 返回契约

按表中顺序判断：

观察情况	返回
原目录结果 status 不是 success	原样返回原失败结果，保留原错误类别
目录结果 truncated=true	{”status”: ”failed”, ”error_type”: ”incomplete_directory_listing”}
完整结果，没有候选文件	{”status”: ”failed”, ”error_type”: ”no_text_file”}
完整结果，有多个候选文件	{”status”: ”failed”, ”error_type”: ”multiple_text_files”}
完整结果，只有一个候选文件	{”status”: ”success”, ”relative_path”: 候选条目的原 relative_path}

候选必须同时满足 type 等于 file、name 以.txt 结尾；扩展名大小写不敏感。目录和符号链接不算本题候选；这不是宣称符号链接一定不可读，而是本题“普通文件”的选择规则。

不要修改输入字典，不要猜路径、拼接项目根目录或去掉合法文件名中的空格。返回已有条目的 relative_path，后续读取工具仍需自己做路径范围检查。

提示：先写提前返回的分支，再遍历 entries 收集候选，最后按候选数量判断。可用字符串 lower 或 casefold 处理扩展名大小写，用 endswith 判断后缀。暂时不提供完整实现。

17.1.4 运行验收

在项目根目录的 Anaconda Prompt 中运行：

```
conda activate agent-study
python -X utf8 -m unittest discover -s exercises -p test_text_file_selection.py -v
```

初始函数主动抛出 NotImplementedError，8 项会报错。这是待完成练习的起点，不代表昨日 Agent 代码回归。只修改 exercises/`select_text_file.py`，不修改验收用例。

AI 备课验证：现有 tests/ 50 项通过；本题 8 项均能加载，初始报错全部来自上述待实现占位。学习者尚未提交实现或运行结果。

完成后再次运行同一命令，应 8 项全部通过。必要时再运行已有功能的测试：

```
python -X utf8 -m unittest discover -s tests -v
```

本题没有新增真实模型实验或结果报告；不要将测试通过记录成已经接入 Agent。通过后，下一步再讨论如何把判断结果接到读取步骤，并区分固定流程和模型决策。

17.1.5 交给我检查

提供你实现的函数和 8 项验收结果，并解释：如果 truncated=true，entries 里恰好只有 hello.txt，为什么不能继续断言“整个目录中有且只有这一个.txt”？

17.1.6 首次提交检查

学习者依据提示补全函数后，AI 用 agent-study 实际运行本题测试：8 项中 5 项通过、3 项失败。未改动学习者函数。问题集中在候选数量判断与成功 return 仍位于 for 循环内部；待学习者将数量判断移至遍历结束之后，再复验。暂未完成验收，也尚未接入 Agent。

17.1.7 修正后复验

2026-10-22 学习者自行调整缩进，AI 检查实际代码并重新运行指定验收命令：8 项全部通过。候选筛选先完成遍历，再判断数量；失败传播、截断优先判断、空目录、多文件、大小写、路径空格与输入不变均符合本题契约。本次保留学习者源码，未代改。截断观察的独立解释尚待复述，函数还未接入 Agent 或真实读取流程。

17.2 第 18 次动手：将选择函数接到真实读取步骤

日期：2026-10-22。前置：学习者依据提示完成 `select_unique_text_file`，并自行修正循环缩进；AI 复验 8 项测试通过。

17.2.1 本次只学一个连接点

```
list_directory 的真实结果
  → select_unique_text_file 判断能否确定唯一文件
```

```
    → 失败：立即返回，不调用读取
    → 成功：把 selection["relative_path"] 交给 read_text_file
      → 返回真实读取结果，可能成功，也可能失败
```

新入口是 labs/l15_directory_read_workflow.py。它直接导入 exercises/select_text_file.py 中学习者已完成的函数，不复制另一份实现。本课为教学连接实验，尚未将练习函数迁入正式工具模块；原有真实模型循环不变。

17.2.2 先看核心代码

```
directory_result = call_tool(
    tools, "list_directory", {"relative_path": relative_directory},
)
selection = select_unique_text_file(directory_result)

if selection["status"] != "success":
    return ...  # 返回失败和已执行轨迹，下面的读取不会运行

read_result = call_tool(
    tools, "read_text_file", {"relative_path": selection["relative_path"]},
)
```

selection 的路径来自刚获得的目录观察，不从用户任务猜文件名，也不用在这里拼接项目根目录。注册表仍绑定可信根目录，call_tool 仍负责参数校验、已知异常转换；读取工具仍负责解析路径并检查范围。

如果目录工具失败，选择函数会原样传递失败，随后流程提前返回。这里采用固定停止规则，不调用全局 decide_tool_result，也没有参数修正或重试预算；不能将这个教学流程的停止行为与原生 Agent 策略混为一谈。

17.2.3 为什么还要检查读取结果

选择成功只说明目录观察完整，且满足唯一普通.txt 的规则；扩展名不保证文件内容是 UTF-8，也不保证列出后文件仍然存在或权限没变。因此最终 status 取自 read_result，而不是把 selection 的 success 当作读取成功。

返回值包含三个部分：

字段	用途
status	整个固定流程成功或失败

字段	用途
result	最后决定结果的原始选择/工具字典，保留 content 或 `error_type`
trace	实际经历的步骤及各自结果；不是模型消息或模型决策

例如读取成功后，正文位于 result["result"]["content"]。最外层 result 是你接收整个流程返回值的变量；里面的"result" 键保存最后一步结果。

17.2.4 运行三个真实案例

```
conda activate agent-study
python -X utf8 -m labs.l15_directory_read_workflow
```

案例	文件来源	应观察到的步骤	最终结果
`unique_text_file`	项目 labs/fixtures	列目录、选择、读取	success，两行真实正文
`multiple_text_files`	自动创建的临时目录，两份.txt	列目录、选择	failed / `multiple_text_files`，没有读取
`invalid_utf8_file`	自动创建的临时目录，真实非法 UTF-8 字节	列目录、选择、读取	failed / `invalid_encoding`，选择曾成功

临时案例结束后自动清理，不修改已有 hello.txt。最后两个 failed 是预期演示结果，不是脚本意外崩溃。报告保存到 reports/`l15_directory_read_workflow`.json；报告中的三个步骤不代表三次模型请求，本实验没有请求模型。

```
python -X utf8 -m unittest discover -s tests -v
python -X utf8 -m unittest discover -s exercises -p test_text_file_selection.py -v
```

新增 6 项连接验收，使用真实临时文件，并观察读取函数是否实际被调用。它们覆盖唯一文件、多个文件、没有文本、真实截断、列目录失败、读取解码失败；不是只检查打印的步骤名称。

AI 备课实测：三个演示符合上表，tests/ 56 项全部通过，第 17 课 8 项也通过。学习者尚未提供本课独立运行结果或解释；原始报告保留为 AI 验证证据。

17.2.5 与 l14 真实 Agent 对照

l15 的执行顺序和分支由 Python 预先写好，属于固定 workflow；没有模型参与选择。l14 则是模型根据任务和观察提出下一步工具请求。二者都调用真实工具，但决定下一步动作的位置不同。本课是把学习者函数连接到读取的验证基线，还不是把它塞进模型循环或注册成第三个工具。

17.2.6 本次理解检查

运行后解释：`multiple_text_files` 案例为什么只有两个 trace 步骤？是哪一个 return 让 `read_text_file` 没有执行？

再观察 `invalid_utf8_file`：选择成功后为什么整个流程仍然失败？先把这两个问题解释清楚，再进入下一步。

17.2.7 学习者运行验收

2026-10-22 学习者独立执行 l15，提供三个案例的完整终端输出，结果均与预期相符；正确解释 selection 失败后的 return 结束整个函数，阻止读取步骤。非法编码案例的独立解释仍待确认。本次输出不包含独立测试运行，不混记为已独立通过 56+8 项测试。

随后学习者正确解释：非法编码时失败来自读取结果，最外层 status 取自 `read_result`["status"]，选择成功不代表读取成功。该理解检查通过；AI 补充了读取工具抛出异常、`call_tool` 将其转换为 `invalid_encoding` 的精确分工。

17.3 第 19 次动手：根据执行证据验收 Agent

日期：2026-10-22。前置：学习者独立运行 l15 三个案例，正确解释提前 return 会阻止读取，并指出最终 status 取自 `read_result`，选择成功不代表读取成功。

17.3.1 本次核心区别

completed 是 `run_ollama_loop` 的结束状态：模型给出了非空最终回答。本次另做一个验收器，判断可信执行记录里是否有本题需要的目录发现和实际读取证据。

```
run = run_ollama_loop(...)
verification = verify_unique_text_read(run, "data")
```

这两个结果分别保存。验收器不把原 run["status"] 改掉，也不让模型自己决定 passed。

结果	解释
run[”status”] == ”completed”	模型循环正常结束
verification[”status”] == ”passed”	本题要求的读取执行证据通过检查
completed + failed	循环结束了，但执行证据不符合本题要求

本课先实现并测试这个独立的结束后检查，不修改原生循环本身，也不增加失败后的自动修复。

17.3.2 看什么证据

实现位于 `experiment_agent`/verification.py，入口 `verify_unique_text_read(run_result, expected_directory)`。目标目录由调用方明确传入，不从模型声称的“目标”中反推。

1. 先检查循环是否 completed。
2. 将 assistant.`tool_calls` 与随后 role=tool 的 JSON 结果按现有串行执行顺序配对；只有请求没有执行结果，不算证据。
3. 找到目标目录的观察，确认它先于读取；若读前有多次目标目录观察，采用读前最近的一次。
4. 复用学习者的 `select_unique_text_file`，要求目录成功、完整，且恰好有一个普通.txt 文件。
5. 核对读取请求里的 `relative_path` 是否就是候选路径，再检查对应的真实工具结果是否 success、是否含字符串 content。

项目相对路径会统一斜杠和./，但不接受绝对路径或..。本题按规范化后的路径字符串比较，不推断操作系统特有的大小写或链接别名等价关系。

17.3.3 先看两个决定性的分支

```
if not reads:
    return fail("missing_read_evidence")
```

模型最终文本“我已读取”不会生成一条真实的 `read_text_file` 工具结果，所以不能绕过这个检查。

```
if actual != target:
    return fail("wrong_file_read", expected_path=target, actual_path=actual)
```

读取另一个项目内文件可能完全合法、工具也可能 success，但它没有满足本题的目标。因此“工具有权限执行”和“任务做对了”必须分开判断。

工具读取结果 status 不是 success 时，验收返回 `read_failed`，并保留工具 `error_type`。即使模型随后说成功，也不能改变已经记录的读取错误。

17.3.4 三个稳定案例

```
conda activate agent-study
python -X utf8 -m labs.l16_agent_task_verifier
```

本实验用确定性的模型替身返回原生消息，通过已有 `run_ollama_loop` 和注册表执行真实临时文件工具。三个案例不调用 Ollama，不代表真实模型本次发生这些错误。

临时项目 data/ 下只有 hello.txt；other/other.txt 是另一个可访问但不符合任务目标的文件。

案例	实际动作	循环状态	验收
`correct_completion`	列 data，读取 data/hello.txt，给出正文	completed	passed / `verified_read`
`claimed_without_reading`	列 data 后直接声称已读取	completed	failed / `missing_read_evidence`
`wrong_file_read`	列 data，成功读取 other/other.txt	completed	failed / `wrong_file_read`

报告 reports/`l16_agent_task_verifier`.json 分别保存 run 和 verification，保留原生消息与真实工具结果。临时文件结束后清理；验收器只读记录，不重新执行文件工具，因此临时项目清理后仍能复核这些记录。

```
python -X utf8 -m unittest discover -s tests -p test_task_verifier.py -v
```

13 项测试还覆盖目录截断、多候选、读取失败、空正文、请求缺少结果、工具结果配错或孤立、目录不符、先读后列、记录格式错误和循环未完成。部分负面记录由测试明确修改以验证拒绝分支；不是历史真实模型故障报告。

AI 备课实测：三个演示符合上表；验收器 13 项、全部 tests/ 69 项、第 17 课 8 项练习均通过。另将已保存的 l14 discover 真实模型报告送入验收器，结果 passed；这是旧记录复核，没有新增模型推理。学习者独立运行与解释尚待确认。

17.3.5 这版验收的边界

本课验的是“完成正确文件的读取”这部分执行证据，尚不评分最终回答是否忠实呈现正文，也不证明之后磁盘内容没有变化。空文本文件的 content=”” 是合法读取，不能仅因正文为空而失败。

该函数依赖可信运行时保存的消息，不是日志防篡改或签名系统；不同供应商的并行调用关联格式不在本课支持范围。当前规则保守：记录中出现其他文件读取或读取失败就不通过，后续若加入恢复与重试，需要另外定义验收契约。

复用学习者选择函数保持任务规则一致，但也意味着验收器共享这部分判断代码；它不是另一套完全独立的目录规则实现。当前选择规则有独立行为测试，后续正式评测还需要可信任务标注和冻结的评测集。

17.3.6 理解检查

运行后比较：`wrong_file_read` 中读取工具返回 success，循环也 completed，为什么验收仍然 failed？是哪两个路径不一致？

先解释这一个问题，再把验收显示接到真实模型实验入口。

17.3.7 学习者验收记录

2026-10-22 学习者独立运行三个案例，输出全部符合预期；指定 13 项验收器测试通过，耗时 0.011 秒。正确解释第三例读错文件：期望 data/hello.txt，实际 other/other.txt，工具成功不代表满足任务目标。本课运行与理解检查通过；验收器主体由 AI 编写，独立实现尚未验收，真实模型入口的验收显示尚未接入。

17.4 第 20 次动手：真实模型入口同时显示循环与验收状态

日期：2026-10-22。前置：学习者独立运行 l16 三个案例及 13 项验收器测试，正确指出目标路径与实际读取路径不同导致验收失败。

17.4.1 连接位置

本课直接更新已有 labs/`l14_multi_tool_agent`.py 的 discover 入口。原 `run_ollama_loop` 负责与本地 Ollama 交互、执行工具和结束；返回后，Python 再验收记录：

```
result = run_ollama_loop(...)

if case == "discover":
    verification = verify_unique_text_read(result, "labs/fixtures")
else:
    verification = {"status": "not_applicable", "reason": "discover_only"}
```

验收放在循环结束后。它不向模型发新请求、不重新执行文件工具、不替模型补调用，也不将原来的 result["status"] 改成 passed/failed。

17.4.2 运行

```
conda activate agent-study
python -X utf8 -m labs.l14_multi_tool_agent --case discover
```

本命令会实际调用本机 Ollama 的 qwen3.5:9b-`q4_K_M`；每次 HTTP 请求的 socket timeout 仍为 180 秒，最多 5 次模型请求。真实输出以模型实际行为为准，不保证每次相同。

如果模型正常完成且证据通过，可看到：

```
Loop status: completed
Task verification: passed
Verification reason: verified_read
Verified path: labs/fixtures/hello.txt
```

如果模型直接声称读取却没有实际读取，则可能是：

```
Loop status: completed
Task verification: failed
Verification reason: missing_read_evidence
```

后一种情况下，原循环仍然是 completed，但实验入口不接受本题结果，保存报告后以非零退出码结束。失败不会自动启动重试。

17.4.3 报告里的两个结果

reports/`l14_multi_tool_agent_discover`.json 同时保存：

```
{
  "result": {"status": "completed", "messages": []},
  "verification": {
    "status": "passed",
    "reason": "verified_read",
    "relative_path": "labs/fixtures/hello.txt"
  }
}
```

这是字段示意，省略了实际消息内容；真正报告保留全部请求、响应和工具结果。`result.status` 是循环状态，`verification.status` 是独立验收状态。

旧的 `expected_sequence_matched` 仍显示工具名称序列是否恰好等于 `list_directory` → `read_text_file`，但对于 discover 它只是诊断信息。模型多做一次合理的目录查看可能使序列不相等，却仍满足读取证据规则；反之名称序列相等也可能读取错文件。因此 discover 的最终接受条件改为循环 completed 且验收 passed。

17.4.4 验收器适用范围

本验收器要求先发现目录再读取目标，不适用于只列目录的 list 或直接读取指定文件的 read。这两个模式显示 `not_applicable` / `discover_only`，仍使用原有循环结束与简单序列检查；不能把 `not_applicable` 当作已通过任务级验收。

当前检查的是“完整目录中有唯一普通.txt，并成功读取”的执行证据。无候选或多候选时正确停止、最终回答是否忠实于正文等其他任务结果，尚未定义通过规则；不能据此声称完整语义评分已经完成。

17.4.5 连接验收测试

```
python -X utf8 -m unittest discover -s tests -p test_real_entry_verification.py -v
```

6 项测试使用确定 HTTP 响应和真实临时文件，覆盖通过、没读取、读错文件、非适用任务、步数耗尽，以及多一次目录调用但证据合格。它们不请求真实模型。

AI 备课检查：新增 6 项通过，项目 tests/ 全部 75 项及选择练习 8 项通过。真实模型运行结果及来源记录见 notes/real-entry-verification-run.md。学习者尚未提供本课独立输出。

AI 已实际运行 discover：3 次模型请求，依次列目录和读取文件，正确返回两行正文；Loop status=completed、Task verification=passed、reason=`verified_read`，路径 labs/fixtures/hello.txt，循环耗时约 15.592 秒。该数值不是学习者独立测量；不与昨日不同运行条件直接比较性能。

17.4.6 理解检查

真实运行后，报告中 result["status"] 和 verification["status"] 分别回答什么问题？如果一个是 completed、另一个是 failed，能否把本题记作读取验收通过？

17.4.7 学习者验收记录

2026-10-22 学习者独立运行真实 discover 任务，3 次模型请求、两次实际工具调用，总耗时 4.993 秒，循环 completed 且验收 passed，目标路径和正文正确。正确解释前者说明循环正常结束、后者说明实际读取证据通过。本课运行与状态区分通过，尚不代表独立实现验收器或已独立运行全部 75 项测试。

17.5 第 21 次动手：受控执行 Python 脚本

日期：2026-10-22。前置：学习者已独立跑通真实模型目录发现与读取任务，并正确区分循环 completed 与读取证据 passed。

17.5.1 本课目标

实现 `run_python_script` 工具，运行三个固定、可信的脚本，记录进程退出码、标准输出、标准错误和超时。先完成真实子进程实验，本课不调用模型、不运行训练、不评分实验指标。

实现：`experiment_agent`/tools/`script_tools`.py。演示：labs/`l17_controlled_python_script`.py。

17.5.2 工具只能接收 script_id

```
tools = build_tool_registry(str(root), enable_python_scripts=True)
result = call_tool(tools, "run_python_script", {"script_id": "normal"})
```

允许的 ID 与脚本路径由 Python 固定：

script_id	固定脚本	行为
normal	labs/fixtures/process_scripts/normal.py	输出计算结果与一条 stderr 诊断，正常结束
error	labs/fixtures/process_scripts/error.py	先输出一句文本，再触发真实 RuntimeError

script_id	固定脚本	行为
timeout	labs/fixtures/process_scripts/timeout.py	刷新启动输出后等待 10 秒，超过工具默认 2 秒时限

调用方不能通过工具参数传入路径、命令、解释器、额外参数或时限。Schema 用 enum 描述固定 ID；运行时仍有实际参数校验和名单判断，不能只依赖 Schema。

默认 build_tool_registry(root) 仍只启用原有读取和目录工具。当前 l14 的真实模型没有获得脚本执行能力；本课入口明确启用它，再按预定顺序调用三个脚本。

17.5.3 看核心执行代码

```
command = [sys.executable, "-I", "-X", "utf8", "-u", str(script)]
completed = subprocess.run(
    command,
    cwd=root,
    shell=False,
    stdin=subprocess.DEVNULL,
    capture_output=True,
    text=True,
    encoding="utf-8",
    errors="replace",
    timeout=timeout_seconds,
    check=False,
)
```

- sys.executable 使用启动工具的同一个 Python；在 agent-study 中运行入口，子进程也使用该环境解释器。
- 传递参数列表，shell=False；不把模型字符串当作 shell 命令执行。
- -I 隔离部分 Python 启动路径与配置，-X utf8 指定 Python UTF-8 模式，-u 及时刷新输出；这些选项不是操作系统沙箱。
- cwd 设置子进程工作目录，stdin=DEVNULL 表示不提供交互输入。
- capture_output 分别捕获 stdout 和 stderr；text/encoding 将输出读成文本。errors=replace 避免输出解码失败丢失整份记录，不能声称任意二进制输出都可无损保存。
- check=False 使普通非零退出作为 CompletedProcess 返回，由本工具读取 returncode 并形成结构化失败。

Python 3.11 subprocess 文档

17.5.4 四个字段，不能混为一谈

字段	含义
stdout	子进程标准输出，不保证打印内容正确
stderr	子进程错误或诊断输出，不保证非空就失败
`exit_code`	子进程退出码；正常案例 0，未捕获异常案例 1
status	本工具按执行结果判断的 success 或 failed，不是模型循环状态或任务验收结果

正常脚本故意向 stderr 写一条诊断，但 `exit_code`=0，所以仍 success。异常脚本在失败前也有 stdout，但 `exit_code`=1，所以仍 failed / `process_failed`。不能只看某个输出流是否为空判断成功。

17.6 子进程里的 RuntimeError 不会直接跳进父工具的 except RuntimeError

异常发生在另一个 Python 进程中。子进程向 stderr 写堆栈并非零退出，父工具得到 CompletedProcess，然后根据 returncode 返回 `process_failed`。不要把 traceback 字符串当作父进程抛出的异常对象。

17.7 超时另走异常分支

subprocess.run 超时会抛出 subprocess.TimeoutExpired，由工具转换为 failed / `tool_timeout`，`timed_out`=true。这里复用之前工具超时的错误类别，但实际异常类型是 TimeoutExpired，不是内置 TimeoutError。

超时异常没有提供本工具可用的退出码，因此 `exit_code` 为 Python None，保存到 JSON 后是 null；不编造 1 或 -9。超时异常里的已捕获输出可能是 bytes，所以辅助函数转成 UTF-8 文本，并处理没有输出的 None。

subprocess.run 会终止并等待它启动的直接子进程，再抛出超时异常；进程创建与调度有额外耗时，因此默认 2 秒不是整个工具调用恰好 2.000 秒的保证。官方超时说明

17.7.1 运行三个真实案例

```
conda activate agent-study
python -X utf8 -m labs.l17_controlled_python_script
```

案例	status	exit_code	timed_out	关键观察
normal	success	0	false	stdout 有计算输出，stderr 有普通诊断
error	failed	1	false	process_failed，stderr 含真实 RuntimeError 堆栈
timeout	failed	null	true	tool_timeout，保留“开始等待”，没有“等待结束”

报告：reports/l17_controlled_python_script.json。后两项 failed 是预期的演示结果，父演示程序正常退出并保存三项记录；不能把父命令的退出码与每个子进程的 exit_code 混用。

```
python -X utf8 -m unittest discover -s tests -p test_script_tools.py -v
```

新增 12 项测试包括真实正常/异常/超时、默认不暴露执行能力、拒绝路径或命令片段、拒绝覆盖配置、允许脚本缺失/为目录/越出根目录，以及明确注入的进程创建失败与超时 bytes 输出。

AI 已实测三个子进程，结果符合上表；tests/ 全部 87 项、exercises/ 12 项均通过。学习者独立运行与解释尚待确认。

17.7.2 当前控制范围

名单只限制入口，脚本内容仍必须可信。它们用当前用户的操作系统权限运行，并继承普通环境变量；本工具没有阻止脚本访问项目外文件或网络，也没有限制 CPU、内存和输出总量。capture_output 在内存中收集输出，本课仅用于已审查的短输出脚本。

当前三个脚本不创建后代进程；本工具没有实现任意进程树清理或完整沙箱。不要把它改成接收任意模型生成代码的执行器。真正的隔离、资源限制与后代进程控制属于后续课程。

17.7.3 本次理解检查

normal 案例的 stderr 明明有内容，为什么 status 仍然 success？error 案例的 stdout 也有内容，为什么 status 仍然 failed？先找出判断依据，再接入模型选择。

17.7.4 学习者验收记录

2026-10-22 学习者独立执行三个真实子进程案例，结果均符合预期，指定 12 项工具测试全部通过（2.138 秒）。正确解释 normal 的 stderr 是诊断信息，退出码为 0 且未超时，故工具判定 success。指定运行与该理解检查通过；不将其记为独立实现工具、已独立通过全部 87 项测试或已接入真实模型。

17.8 第 22 次动手：真实模型选择脚本，策略控制继续或停止

日期：2026-10-22。前置：学习者独立运行三个真实子进程案例和 12 项工具测试，正确解释 stderr 可以是诊断信息，正常结束的退出码才是工具 status 的判断依据。

17.8.1 本次分工

```
用户任务 + 三个脚本 ID 的说明
  → 模型提出 run_python_script(script_id=...)
  → Python 校验参数与调用次数，启动固定脚本
  → 保存 role=tool 的真实执行结果
  → Python 策略判断
      success：continue → 下一次模型请求 → 模型根据结果总结
      非零退出或超时：stop → 返回失败与原因，不再请求模型
```

模型负责选择脚本并在成功后总结。必须停止的执行错误由程序策略保证，不依赖模型是否愿意停止。失败结果保存在 messages 中，但 stop 后没有下一次 chat，因此失败结果本轮没有再次发送给模型；终端显示的失败原因是 Python 打印，不是模型看完失败后生成的回复。

17.8.2 新入口，不改变原只读入口

入口 labs/l18_real_script_agent.py 显式启用执行能力，再仅把 run_python_script 放进本轮工具表和工具说明。l14 默认仍提供原来的两个只读工具，不能把 l18 的能力范围理解为所有入口都能执行脚本。

```
registered = build_tool_registry(str(root), enable_python_scripts=True)
# 本入口另加一次调用上限，再暴露该包装函数。
```

```
tools = {"run_python_script": run_once}
result = run_ollama_loop(task, chat, tools, descriptions,
                         max_steps=4, tool_policy=decide_tool_result)
```

Schema 向模型说明 normal/error/timeout 三个 ID，模型自行填写 `script_id`。Python 中的 `EXPECTED_SCRIPT` 只在运行后核对，不参与模型参数生成，也不会把错误选择偷偷替换为正确脚本。

17.8.3 三个真实模型任务

```
conda activate agent-study
python -X utf8 -m labs.l18_real_script_agent --case normal
python -X utf8 -m labs.l18_real_script_agent --case error
python -X utf8 -m labs.l18_real_script_agent --case timeout
```

不带参数默认 normal；–case all 依次运行三个独立会话。每个任务重新设置执行调用预算，不共享上一个案例的消息。

case	模型应选择	执行结果	策略与循环预期
normal	normal	success / `exit_code`=0	continue，再请求模型总结，completed
error	error	failed / `process_failed` / `exit_code`=1	stop / `script_process_failed`，循环 failed
timeout	timeout	failed / `tool_timeout` / `exit_code`=null	stop / `tool_timeout_needs_review`，循环 failed

工具失败时不再请求模型，所以 error/timeout 通常各只有一次模型请求；若先修正过错误参数，次数可能更多，以实际记录为准。正常案例通常两次：选择执行、根据结果回答。

17.8.4 几种限制分别控制什么

限制	本课设置	控制对象
`max_steps`	4	每个任务最多几次模型请求

限制	本课设置	控制对象
`max_corrections`	默认 1	参数校验失败后允许模型修正的次数
`execution_calls`	最多 1	每个任务最多调用一次固定脚本执行函数
脚本 timeout	2 秒	子进程等待时限
HTTP timeout	180 秒	单次本地模型 HTTP 请求等待

参数结构错误在分发器处拒绝，不消耗执行函数调用机会；第一次结构合法调用进入执行函数后就消耗机会，即使 ID 不被允许或进程没能启动也不补回。额外执行请求返回 `execution_limit_exceeded`，策略给出 `script_execution_budget_exceeded` 并停止。此限制是 Python 代码执行的，不只是提示模型“不要重试”。

原有 `run_ollama_loop` 无需修改。只给策略补充 `process_failed`、`process_start_failed`、`script_not_allowed`、`script_not_file` 和 `execution_limit_exceeded` 的明确停止原因；未知错误仍按原默认停止。

17.8.5 怎样读输出

正常案例应有：

```
Loop status: completed
Model decisions: 2
Execution function calls: 1
Policy: continue / tool_succeeded
Final model reply: ... 模型根据真实结果给出的总结...
Expected behavior matched: True
```

异常案例应有：

```
Loop status: failed
Model decisions: 1
Execution function calls: 1
Policy: stop / script_process_failed
No final model reply: loop stopped before requesting another reply.
Expected behavior matched: True
```

这是典型输出示意，不保证每次模型请求次数相同。Expected behavior matched=True 表示实际脚本选择与控制流符合本案例预期；故障演示正确停止也可以 True。它不表示异常脚本执行成功，也不是实验指标评分或最终回答语义验收。

报告分别保存到 reports/l18_real_script_agent_normal.json、_error.json、_timeout.json，包含完整请求、响应、工具结果、tool_decisions、执行函数调用次数与预期行为核对结果。若选错脚本、缺少执行或不符合预期，入口保存报告后非零退出，不自动重跑模型以隐藏失败。

17.8.6 连接测试

```
python -X utf8 -m unittest discover -s tests -p test_real_script_agent.py -v
```

新增 8 项测试只替换 HTTP 响应，真实执行教学脚本。覆盖成功结果进入下一次请求、异常/超时立即停止、选错允许脚本、修正参数、同批重复执行被阻止、其他工具不可用，以及脚本已成功但模型步数耗尽。

AI 备课检查：tests/ 全部 95 项、exercises/ 12 项通过。真实模型运行记录与学习者验收来源见 notes/real-script-agent-run.md；本课实际模型输出不能用测试中的确定响应替代。

AI 已实际运行三个模型任务，脚本 ID 均选择正确：normal 两次请求、循环 completed；error 与 timeout 各一次请求并由策略停止、循环 failed。三个 Expected behavior matched 均 True，耗时分别约 50.487、0.360、2.331 秒。这些是 AI 观测，不是学习者独立运行证据，也不把不同加载条件下的时长当作性能优化结论。

17.8.7 理解检查

error 案例的工具失败结果是否已经发给模型，再由模型决定停止？还是只有消息保存，随后 Python 策略立即结束循环？结合 Model decisions、tool_decisions 和最后的消息角色回答。

17.8.8 学习者验收记录

2026-10-22 学习者独立运行三个真实模型任务，选择和预期行为均正确，请求次数 2/1/1；指定 8 项连接测试通过（2.211 秒）。正确解释失败观察先保存，然后策略停止，没有下一次模型调用，因此模型未看到该失败结果。本课运行与理解检查通过，尚不代表独立实现了该入口或已独立执行全部 95 项回归测试。

17.9 第 23 次动手：自己组织 Agent 的核心循环

日期：2026-10-23。当前状态：学习者有提示尝试后，AI 按请求修复实现，8 项验收通过；下文保留原始练习契约。控制流解释及独立迁移检查待完成，G1 尚未正式通过。

前面的工具、策略和真实模型入口已经能运行。这次由你实现核心控制流程，检查是否能把“理解运行结果”迁移到“组织运行过程”。这是 G1 的代码部分；测试通过后还要解释控制流并完成一次小修改，才讨论正式通过 G1。

17.9.1 本次写哪里

补全 `exercises/g1_agent_loop.py` 中的 `run_agent`。不要改验收测试，也不要调用 `experiment_agent/loop.py` 或 `ollama_loop.py` 中的完整循环代替实现。先依据下面的契约自己写；卡住时再逐步索取提示。

可以直接复用已学的两个函数：

- `call_tool(tools,name,arguments)`：查工具、验证参数、执行并转换已知异常。
- `decide_tool_result(result,corrections_used)`：返回包含 `action`、`reason` 的策略字典。默认整个任务只允许一次参数修正。

本次不请求 Ollama、不执行子进程。测试用可预测的模型替身及真实临时文件。这样可以把注意力放在循环和消息上，不受推理耗时影响。

17.9.2 输入和消息契约

函数签名：

```
run_agent(task, model, tools, tool_descriptions, max_steps=5)
```

每轮调用 `model(messages,tool_descriptions)`，获得一个动作：

```
{"type": "tool_call", "name": "read_text_file",
 "arguments": {"relative_path": "data/notes.txt"}}
# 或
{"type": "final", "content": " 回答正文"}
```

这里沿用 l06 的教学协议，消息的 `content` 可以直接保存字典；它不是 l11/l18 的 Ollama 原生 HTTP 消息格式。

消息来源	保存格式
用户	`{"role": "user", "content": task}`
模型动作	`{"role": "assistant", "content": action}`
工具结果	`{"role": "tool", "name": name, "content": result}`

开始时只有用户消息。每轮先请求模型，再记录本轮动作；即使动作非法，也记录原值后返回诊断。工具结果必须先保存，再执行策略分支。`append` 只保存消息，下一次调用 `model(...)` 时模型才收到更新后的列表。

有效 `final` 必须有非空白字符串 `content`，回答保留原字符串。有效 `tool_call` 必须有非空白字符串 `name` 和 `arguments` 字段；参数内容交给分发器验证。其余动作是 `invalid_model_response`。模型回调自身抛出异常的恢复、本次教学协议以外的原生消息解析，不属于这道练习。

17.9.3 返回值契约

所有正常返回的字典都包含 `status`、`steps`、`messages`。`steps` 表示已经请求模型的次数，第一个动作是第 1 步。骨架已提供 `max_steps<1` 的检查；本题假定调用方提供整数步数。

情况	必须返回的其他字段
有效 final	`status="completed"`，`answer=` 回答原文
非法模型动作	`status="failed"`，`error_type="invalid_model_response"`
策略要求停止	`status="failed"`，`error_type="tool_policy_stopped"`，`tool_error_type=result.get("error_type")`，`stop_reason=decision["reason"]`
决策次数耗尽，仍无 final	`status="failed"`，`error_type="max_steps_exceeded"`

工具成功不等于循环完成，仍需要模型下一次给出 final。达到最后一轮时也要执行该轮请求的工具并保存结果，然后才能报告步数耗尽。

策略的三种动作：

- `continue`：工具成功，进入下一轮。
- `correct`：累计消耗一次修正机会，再进入下一轮；之后工具成功也不能重置这个计数。
- `stop`：立即返回，保留刚保存的工具结果，不再请求模型。

策略停止与步数限制是两种退出原因。即使还有剩余步数，缺失文件等错误也应立即停止。

17.9.4 八项验收

1. 模型直接 final：只请求一次，不执行工具。
2. 列目录 → 按观察到的路径读取 → final：后续模型确实收到前面的工具结果。
3. 参数名错误 → 正确参数 → final：允许一次修正。
4. 错误 → 成功 → 再错误：修正预算不因成功而重置。
5. 文件缺失：结果已保存，但策略停止后不再请求模型。
6. 未知工具：停止，不替换成别的已注册工具执行。
7. 两次工具成功但没有 final：决策预算耗尽，最后结果仍保留。
8. 非法动作：记录诊断，不执行工具；包含多个子案例。

在已激活的 Anaconda 环境运行：

```
cd /d E:\study\agent
conda activate agent-study
python -X utf8 -m unittest discover -s exercises -p test_g1_agent_loop.py -v
```

初始骨架会因 `NotImplementedError` 报错，这是待实现的练习状态。先不要用是否全绿衡量第一小步；全部补完后，目标是 `Ran 8 tests ... OK`。这些测试位于 `exercises/`，不改变产品回归测试 `tests/` 的范围。

17.9.5 第一小步

先写初始化：`messages` 里只放用户任务，`corrections_used` 从 0 开始。再写最多 `max_steps` 次的循环，每轮取得模型动作并保存 assistant 消息。然后考虑 final 分支应该返回什么，为什么这里的 return 会结束整个函数。

先完成这一小段即可继续讨论。工具执行、策略判断和超限退出留待后续逐步补上；不要为了让不完整的中间版本过测试而修改测试。

代码完成后，复述一次“保存观察”和“发送观察”的区别，并在临时案例中说明最后一次工具成功但没有 final 时如何退出。这两项检查尚未发生，不计为已掌握。

17.10 第 24 次动手：进程成功与实验指标验收

日期：2026-10-23。状态：学习者在提示后补全函数，AI 代码检查及指定 8 项测试通过；新增数值规则的独立解释与实际文件连接待验收。下文保留原始练习任务。

17.10.1 新知识点

此前我们区分了循环 completed 与读取任务 passed。现在把任务验收迁移到代码实验：子进程正常退出，只说明没有通过非零退出码或超时报告失败；实验任务还需要检查实际产物和指标。

例如任务要求：生成 metrics.json，其中 mse 必须是有限的非负数值，且不超过 0.1。

下面是概念示例，不是真实训练测量：

例子	执行观察	产物观察	任务判断
A	`exit_code`=0，`timed_out`=false，工具 success	文件已读取，JSON 为 {”mse”: 0.04}	指标规则可通过
B	`exit_code`=0，`timed_out`=false，工具 success	预期文件没有生成，只有 stdout 声称已保存	缺少产物，不能通过
C	`exit_code`=0，`timed_out`=false，工具 success	文件已读取，JSON 为 {”mse”: 1.2}	数值合法，但未达到阈值

三个层级的职责：

1. 执行工具：记录进程退出码、超时和输出。
2. 读取与解析：确认指定产物可读取，且是有效 JSON。
3. 指标验收：确认数据结构、数值合法性及是否达到任务阈值。

本课只实现第 3 层，输入已经是解析后的数据，不新增脚本执行权限，不训练模型。下次再连接实际文件读取与执行记录。单独通过指标函数不能证明文件存在、来自本次运行或确实经过训练；这些证据需要由外层验收补齐。

17.10.2 小练习：verify_metrics

编辑 exercises/`verify_metrics`.py，补全：

```
verify_metrics(metrics, max_mse=0.1)
```

metrics 是 JSON 解析后的值，可能是字典，也可能是列表、字符串或 None。`max_mse` 来自可信 Python 调用方，本题假定它是有限非负数值，不从模型回答中取阈值。

条件	返回值
不是字典，或没有 mse	{"status": "failed", "reason": "invalid_metrics"}
mse 不是 int/float，或者是 bool	同上
mse 是 NaN、正负无穷或负数	同上
mse 合法，但大于 max_mse	{"status": "failed", "reason": "mse_above_threshold", "mse": 原值, "max_mse": 阈值}
mse 合法，且不大于 max_mse	{"status": "passed", "reason": "verified_metrics", "mse": 原值}

不修改输入，允许额外字段，不自动把字符串"0.04" 转成数字。0 和恰好等于阈值的值都应通过。

注意两个 Python 细节：

- bool 是 int 的子类，单用 isinstance(value, (int, float)) 会接受 True 和 False，必须显式排除 bool。
- NaN 不能通过普通大小比较可靠地识别；在类型检查后使用 math.isfinite(value) 检查有限性。本练习使用常规范围的 int/float。

17.10.3 第一小步

先判断 metrics 是否为字典，再判断是否存在 mse 字段。只有通过这两项检查，才可以取 metrics["mse"]。先不要写执行脚本或解析 JSON 的代码。

完整实现后运行：

```
python -X utf8 -m unittest discover -s exercises -p test_metrics_verifier.py -v
```

共有 8 项测试，包含多个边界子案例。初始骨架的 NotImplementedError 是待实现状态，不计为测试通过。测试只检查上述函数契约，不证明实验已实际运行。

G1 的核心循环由 AI 按请求修复并验收，独立迁移检查仍未完成；本课不把此前 G1 自动标记为通过。

17.11 第 25 次动手：读取真实指标文件并验收

日期：2026-10-23。学习者自己的 verify_metrics 保持不变，AI 新增连接流程；学习者已独立运行五例、指定 8 项测试通过，并正确解释解析失败后的提前返回。独立实现尚未验

收。

17.11.1 数据流

```
相对文件路径
  → call_tool 调用 read_text_file
  → 成功时取得 UTF-8 字符串
  → json.loads 转成 Python 数据
  → verify_metrics 检查结构、数值和阈值
  → passed / failed
```

连接函数位于 experiment_agent/metrics_workflow.py：

```
run_metrics_file_workflow(tools, relative_path, max_mse=0.1)
```

max_mse 是可信调用方设置的任务要求，不由指标文件改变。流程复用原注册表和路径边界，未添加执行能力。

17.11.2 第一步：读文件

```
read_result = call_tool(tools, "read_text_file", {"relative_path": relative_path})
```

读取失败时，立即返回 failed，最后 result 保留原工具错误，不解析 JSON、不验收指标。缺失文件是 file_not_found；非法编码由分发器转换为 invalid_encoding。

17.11.3 第二步：把字符串解析成数据

```
try:
    metrics = json.loads(read_result["content"])
except json.JSONDecodeError as error:
    # 记录 invalid_json 和行列位置，返回失败。
```

read_result["content"] 是文件正文字符串；metrics 才是解析后的 Python 数据。json.loads 接收字符串，json.load 接收已打开的文件对象；这里正文已经由受控工具读出，因此用 loads。

JSON 语法错误与指标不合法分别处理：

- {"mse": 0.04,} 尾逗号导致 JSONDecodeError，流程在解析阶段停止。

- `{"mse": "0.04"}` 是可解析的 JSON，但 mse 是字符串，指标验收返回 `invalid_metrics`。
- `[]`、null 等也可以被解析，随后仍要检查是否满足指标结构。

17.11.4 第三步：调用学习者的验收函数

```
verification = verify_metrics(metrics, max_mse=max_mse)
```

最终流程状态取自 verification["status"]，不能取自读取结果。读取与解析都成功，指标仍可能因为超过阈值而 failed。

trace 保存实际执行过的阶段：

情况	最后结果	trace 阶段
合法且达标	passed / `verified_metrics`	`read_text_file` → `parse_json` → `verify_metrics`
文件缺失	failed / `file_not_found`	`read_text_file`
JSON 语法错误	failed / `invalid_json`	`read_text_file` → `parse_json`
指标类型错误	failed / `invalid_metrics`	全部三阶段
指标超阈值	failed / `mse_above_threshold`	全部三阶段

工具和解析阶段使用 success/failed，指标及整个验收流程使用 passed/failed。没有模型参与，所以没有 Loop status；这是固定 workflow。

17.11.5 在 Anaconda 中运行

```
cd /d E:\study\agent
conda activate agent-study
python -X utf8 -m labs.l19_metrics_file_workflow
python -X utf8 -m unittest discover -s tests -p test_metrics_file_workflow.py -v
```

演示创建真实临时文件并清理，包含上表五例。数据是人工构造的教学样例，不是真实训练结果。报告保存到 reports/`l19_metrics_file_workflow`.json；临时项目绝对路径中的本机信息替换为 <demo_project>。

8 项连接测试还覆盖越界、非法 UTF-8、可信阈值传递及非对象 JSON。此流程只证明指定文件当前可读取且指标满足规则，尚不证明文件来自本次运行、数值由实际训练计算，或完成了训练任务。之后再加入执行与产物来源关联。

17.11.6 本次理解检查

invalid_json 案例里，读取已经 success。为什么流程仍然 failed，而且 trace 里没有 verify_metrics？观察提前 return 的位置，再独立解释。

17.12 第 26 次动手：执行脚本，只验收本次运行的指标

准备日期：2026-10-23。2026-10-25 学习者已在 agent-study 独立运行五例，指定 12 项测试通过，并正确复述本次目录缺少指标才导致外层失败。运行器独立实现与目录绑定机制的完整复述仍待验收。

17.12.1 为什么只读 metrics.json 不够

昨天的实验可能已生成一个合格文件，今天的脚本却没有写文件。如果仍读取共享路径，就会把旧结果当成本次结果。stdout 声称保存成功、文件存在或修改时间较新，都不能单独证明来源。

本次使用新目录和运行编号组合：

```
Python 创建 run_id 和全新 runs/<run_id>/
  → 固定脚本在此目录作为 cwd 的子进程中执行
  → 脚本写入 metrics.json，正文包含同一个 run_id
  → 确认进程 success，否则立即停止
  → 只在本次目录内读取 metrics.json
  → 解析 JSON
  → 比较文件 run_id 与父进程保存的编号
  → 调用已有 verify_metrics
```

17.12.2 一、父进程分配运行上下文

experiment_agent/experiment_workflow.py 中的 run_experiment_workflow 使用 uuid4().hex 生成编号，构造 runs/<run_id>/metrics.json。mkdir(..., exist_ok=False) 拒绝复用旧目录；即使编号碰撞，也报 run_setup_failed，不覆盖旧文件。

路径由可信 Python 程序决定，不接收模型提供的输出位置。runs/ 中的生成物留在本地，并已加入 Git 忽略。

17.12.3 二、脚本在哪写文件

只执行固定的 labs/fixtures/experiment_scripts/write_metrics.py，使用当前 sys.executable，shell=False，stdin 禁用、输出捕获、默认超时 2 秒。cwd 设为本次

运行目录，而不是改变父进程的当前目录。

脚本收到 –run-id 参数；相对写入 Path(”metrics.json”)，因此落在 runs/<run_id>/metrics.json：

```
{"run_id": " 父进程生成的编号", "mse": 0.04}
```

mse=0.04 是固定教学值，没有训练发生。本次脚本执行器是可信工作流内部能力，不注册给模型；原 l17/l18 的三个脚本、注册表和参数限制保持原样。

17.12.4 三、执行失败时不读取结果

即使异常脚本或超时脚本已写入文件，执行失败也立即返回，不做指标验收。不能只因为有文件，就忽略失败的进程。

17.12.5 四、只读本次目录，并检查编号

读取工具的 `project_root` 缩小为本次目录，输入固定为 metrics.json。因此常规路径解析也不能把本次目录外的旧文件当成目标。

已有 `run_metrics_file_workflow` 增加可选的 `expected_run_id` 参数。省略时仍是 l19 的三步流程；传入时，在 JSON 解析成功后、指标验收前，增加 `check_run_id`。

编号缺失或不一致返回 `run_id_mismatch`，不能继续验收 mse，也不去其他目录查找替代文件。通过编号检查后，仍需要数值和阈值检查。

这一关联建立在固定可信脚本、没有外部并发篡改运行目录的前提下。`run_id` 是关联标记，不是签名；它不证明数值来自真实训练，也不阻止恶意脚本主动伪造标记。本课不是完整文件系统或网络隔离沙箱。

17.12.6 五个真实运行案例

案例	执行	预期验收
normal	正常写入本次编号和指标	passed / `verified_metrics`
`missing_metrics`	退出码 0，但不写本次文件；上一例文件仍存在	failed / `file_not_found`，禁止回退读取旧结果
`wrong_run_id`	正常写入合法 mse，但编号故意为 `previous_run`	failed / `run_id_mismatch`
error	写入后抛异常	failed / `process_failed`，不读取文件

案例	执行	预期验收
timeout	写入后长时间等待	failed / `tool_timeout`，不读取文件

在 Anaconda 中运行：

```
cd /d E:\study\agent
conda activate agent-study
python -X utf8 -m labs.l20_run_scoped_metrics
python -X utf8 -m unittest discover -s tests -p test_experiment_workflow.py -v
```

演示保留 runs/ 下的实际产物，报告写到 reports/`l20_run_scoped_metrics`.json。报告把项目根目录替换为 <`demo_project`>；运行编号与项目相对路径保留，便于复盘。12 项测试覆盖新目录、旧文件误用、编号错误、进程失败、超时、阈值、碰撞与拒绝非法执行输入。

17.12.7 本次理解检查

`missing_metrics` 的 execution 明明是 success，旧文件也仍然存在，为什么仍然返回 `file_not_found`？请沿着 `metrics_path` 和读取工具绑定的根目录解释，而不是只回答“因为策略要求停止”。

17.13 第 27 次动手：真实线性回归训练与指标验收

日期：2026-10-25。AI 已实现、运行并测试；学习者已独立运行两组训练及指定 7 项测试，正确区分进程成功与指标未达标。模型配置决策及独立实现尚未验收。

17.13.1 这次变化

之前 `write_metrics`.py 写入固定 mse=0.04，用来验证运行来源和故障分支。本课新增 experiments/`linear_regression`/train.py，真实更新线性模型的参数，再用验证集计算 MSE。原 l20 故障演示保留。

不需要新依赖：使用当前 Anaconda 解释器、CPU 和 Python 标准库。训练脚本仍由固定 workflow 执行，尚未让模型选择配置。

17.13.2 实验规则预先固定

数据由 y=3x+2+ 噪声生成，x 在 [-1, 1] 均匀采样，噪声标准差为 0.1。128 个训练样本、64 个独立生成的验证样本，分别使用固定随机种子 20261025 和 20261026。

两组配置都从 weight=0、bias=0 出发，执行 100 次全批量梯度下降，只改变学习率：0.1 与 0.01。数据、初始化、预算和验收阈值相同；验证集不参与梯度更新、早停或选择训练步数。此处不是最终保留测试集评测。

17.13.3 训练脚本负责什么

模型预测为 prediction = weight * x + bias。对训练集残差求梯度，更新 weight、bias；更新结束后分别计算训练集和验证集 MSE。

```
model = fit_line(train_data, config["learning_rate"], config["steps"])
validation_mse = mse(validation_data, model["weight"], model["bias"])
```

metrics.json 的 mse 字段等于 `validation_mse`，保持与学习者 `verify_metrics` 的接口兼容。mse 由实际预测与标签计算，不能从模型回答、预期答案或阈值反推出一个数值。

额外记录 `initial_train_mse`、`initial_validation_mse` 和 `baseline_validation_mse`。均值基线用训练集标签均值作为常数预测，再在验证集上评估，不使用验证集标签均值拟合基线。

17.13.4 运行器与验收器负责什么

`experiment_workflow.py` 新增两个固定训练 case：`linear_regression`、`linear_regression_slow`。它们指向固定训练脚本，脚本只接受本程序生成的 `run_id` 和固定 case。原执行工具、模型注册表和旧故障 case 不变。

运行仍沿用：

```
新 run_id / 新目录
  → 真实训练子进程
  → 成功退出后读取本次 metrics.json
  → 解析 JSON
  → check_run_id
  → verify_metrics 检查验证集 MSE 是否不超过 0.1
```

训练脚本计算指标，运行器记录执行和路径，验收器判断任务要求是否满足。进程 success 与指标 passed 仍然分别判断。

17.13.5 本次目录保存哪些产物

文件	内容
config.json	学习率、步数、样本数、种子、初始化及数据生成设置
data.json	本次实际训练和验证数据
model.json	训练得到的 weight、bias
`validation_predictions`.json	验证样本的 x、真实标签和预测值
metrics.json	本次 `run_id`、各项 MSE、模型、配置、数据指纹、训练脚本 SHA256、Python 版本及训练耗时

这些文件留在 runs/<`run_id`>/，Git 忽略；精简实验报告保存到 reports/`l21_real_linear_regression`.json。训练耗时只测参数更新循环，区别于运行器的整个子进程耗时。一次极短 CPU 耗时不用于性能排名。

独立测试从 data.json 的验证集与 model.json 重新计算预测和 MSE，核对预测文件及报告，而不调用训练脚本里的 mse 函数。这提供本实验的数值交叉检查；运行时的通用 `verify_metrics` 仍只验结构、有限性、非负性与阈值，不是通用防伪裁判。

17.13.6 实际测量（AI 首次运行）

学习率	步数	初始训练 MSE	最终训练 MSE	最终验证 MSE	验收
0.1	100	6.77084887	0.01132177	0.01207715	passed
0.01	100	6.77084887	0.89226171	0.87744596	failed / `mse_above_threshold`

两组初始验证 MSE 均为 5.57655214，均值基线验证 MSE 均为 3.19075572。学习率 0.1 训练得到 weight≈2.99633、bias≈2.00068。

两组进程都正常退出，`run_id` 均匹配。低学习率配置在固定预算下未达到目标，不自动重试、不改阈值、不伪造成功。此结论只针对该固定小型合成任务及预算，不表示学习率越大总是越好。

17.13.7 在 Anaconda 中运行

```
cd /d E:\study\agent
conda activate agent-study
python -X utf8 -m labs.l21_real_linear_regression
python -X utf8 -m unittest discover -s tests -p test_linear_regression_training.py
↪  -v
```

演示命令保存两组结果。两组都有有效训练、来源关联和对照条件时，命令正常结束，即使其中一组指标未达标；不要把这个演示命令的退出状态理解为每组任务都 passed。

新增 7 项测试包含真实训练、保存产物独立复算、公平对照、重复运行复现、已知无噪声直线、手算 MSE 以及正常执行但指标未达标。此前 G1 独立迁移仍未验收，不因开始训练自动标记为通过。

17.13.8 本次理解检查

`linear_regression_slow` 的 `exit_code` 为什么是 0，Workflow status 却是 failed? 请说明失败来自哪一层、哪个实际指标，不要将其解释成 Python 异常。

18 附录 B. 当前核心源码全文

这些是固定快照中的真实实现；局部片段不能脱离函数层级直接粘贴。全部测试和其他实现见资料索引。

18.1 exercises/select_text_file.py

```
""" 独立练习：根据目录观察选择唯一的普通 .txt 文件；不访问文件或模型。"""


def select_unique_text_file(directory_result: dict) -> dict:
    """ 返回可读取的 relative_path，或返回说明无法选择的结构化失败。

    本题假设输入符合现有 list_directory 的返回结构。
    具体契约见 lessons/17-directory-observation-exercise.md。
    """
    if directory_result["status"] != "success":
        return directory_result

    if directory_result["truncated"]:
        return {
```

```
        "status": "failed",
        "error_type": "incomplete_directory_listing",
    }

    candidates = []

    for entry in directory_result["entries"]:
    # 同时检查:
    # entry["type"] 是否等于 "file"
        if entry["type"] == "file" and entry["name"].lower().endswith(".txt"):
            candidates.append(entry)
    # entry["name"].lower().endswith(".txt") 是否成立
    if len(candidates) == 0:
        return {
        "status": "failed",
        "error_type": "no_text_file",
    }
    if len(candidates) > 1:
        return{
                "status": "failed",
                "error_type": "multiple_text_files",
            }
    # 返回 failed, error_type 为 multiple_text_files
    # 满足时, 用 candidates.append(entry) 保存整个条目
    return {
    "status": "success",
    "relative_path": candidates[0]["relative_path"],
}
```

18.2 labs/l15_directory_read_workflow.py

```
""" 列目录、选择唯一文本文件、读取正文: 固定 Python 流程, 不请求模型。"""

import json
from pathlib import Path
from tempfile import TemporaryDirectory

from exercises.select_text_file import select_unique_text_file
from experiment_agent.tools.registry import build_tool_registry, call_tool


def run_directory_read_workflow(tools: dict, relative_directory: str) -> dict:
```

```
    """ 复用学习者的选择函数；选择失败就返回，不执行读取。"""
    directory_result = call_tool(
        tools, "list_directory", {"relative_path": relative_directory},
    )
    trace = [{"step": "list_directory", "result": directory_result}]

    selection = select_unique_text_file(directory_result)
    trace.append({"step": "select_unique_text_file", "result": selection})
    if selection["status"] != "success":
        return {"status": "failed", "result": selection, "trace": trace}

    read_result = call_tool(
        tools, "read_text_file", {"relative_path": selection["relative_path"]},
    )
    trace.append({"step": "read_text_file", "result": read_result})
    return {"status": read_result["status"], "result": read_result, "trace": trace}


def run_case(label, tools, relative_directory):
    result = run_directory_read_workflow(tools, relative_directory)
    print(f"\nCase: {label}")
    for event in result["trace"]:
        print(f"Step: {event['step']}")
        print(json.dumps(event["result"], ensure_ascii=False, indent=2))
    print(f"Workflow status: {result['status']}")
    if result["status"] == "success":
        print(f"Actual file content:\n{result['result']['content']}")
    else:
        print(f"Stopped because: {result['result']['error_type']}")
    return {"case": label, **result}


def main():
    root = Path(__file__).resolve().parents[1]
    print("Fixed Python workflow; no model API called.")
    print("Case 1 reads the project fixture; other cases use temporary files.")
    reports = [run_case("unique_text_file", build_tool_registry(str(root)),
    ↪  "labs/fixtures")]

    with TemporaryDirectory(prefix="directory-read-lesson-") as directory:
        demo_root = Path(directory)
        multiple = demo_root / "multiple"
        multiple.mkdir()
        (multiple / "a.txt").write_text("first", encoding="utf-8")
```

```
        (multiple / "b.txt").write_text("second", encoding="utf-8")
        invalid = demo_root / "invalid_utf8"
        invalid.mkdir()
        (invalid / "bad.txt").write_bytes(b"\xff\xfe")
        tools = build_tool_registry(str(demo_root))
        reports.append(run_case("multiple_text_files", tools, "multiple"))
        reports.append(run_case("invalid_utf8_file", tools, "invalid_utf8"))

    report_path = root / "reports" / "l15_directory_read_workflow.json"
    report_path.parent.mkdir(parents=True, exist_ok=True)
    report_path.write_text(
        json.dumps(reports, ensure_ascii=False, indent=2) + "\n", encoding="utf-8",
    )
    print(f"\nSaved report: {report_path}")


if __name__ == "__main__":
    main()
```

18.3 experiment_agent/verification.py

```
""" 根据可信运行时的原生消息记录验收目录发现与文件读取；不执行工具。"""

from collections import deque
import json
from pathlib import PurePosixPath

from exercises.select_text_file import select_unique_text_file


def _relative_path(value):
    """ 比较项目相对路径，接受 Windows 分隔符和 ./，拒绝越级或绝对路径。"""
    if not isinstance(value, str) or not value.strip() or ":" in value:
        return None
    path = PurePosixPath(value.replace("\\", "/"))
    if path.is_absolute() or ".." in path.parts:
        return None
    return path.as_posix()


def _tool_events(messages):
    """ 按现有运行时的串行顺序，将请求与真实 tool 结果配对。"""
```

```
    if not isinstance(messages, list):
        raise ValueError("messages must be a list")
    pending = deque()
    events = []
    for index, message in enumerate(messages):
        if not isinstance(message, dict):
            raise ValueError("invalid message")
        role = message.get("role")
        if role == "assistant":
            if pending:
                raise ValueError("tool results missing before next assistant
                ↪  message")
            calls = message.get("tool_calls", [])
            if calls is None:
                calls = []
            if not isinstance(calls, list):
                raise ValueError("invalid tool_calls")
            for call in calls:
                function = call.get("function") if isinstance(call, dict) else None
                if (not isinstance(function, dict)
                        or not isinstance(function.get("name"), str)
                        or not isinstance(function.get("arguments"), dict)):
                    raise ValueError("invalid tool request")
                pending.append(function)
        elif role == "tool":
            if not pending:
                raise ValueError("tool result has no matching request")
            function = pending.popleft()
            if message.get("tool_name") != function["name"]:
                raise ValueError("tool result does not match request order")
            if not isinstance(message.get("content"), str):
                raise ValueError("tool content must be serialized JSON")
            observation = json.loads(message["content"])
            if not isinstance(observation, dict) or observation.get("status") not
            ↪  in ("success", "failed"):
                raise ValueError("tool result must be an object with a known
                ↪  status")
            events.append({"name": function["name"], "arguments":
            ↪  function["arguments"],
                           "result": observation, "message_index": index})
        elif role not in {"system", "user"} or pending:
            raise ValueError("invalid message order")
    if pending:
        raise ValueError("tool request has no execution result")
```

```
    return events


def verify_unique_text_read(run_result: dict, expected_directory: str) -> dict:
    """ 验收完成的运行是否有完整目录观察和正确读取证据；不评价最终回答语义。"""
    def fail(reason, **details):
        return {"status": "failed", "reason": reason, **details}

    expected_directory = _relative_path(expected_directory)
    if expected_directory is None:
        raise ValueError("expected_directory must be a project-relative path")
    if not isinstance(run_result, dict):
        return fail("invalid_execution_trace")
    if run_result.get("status") != "completed":
        return fail("loop_not_completed")
    try:
        events = _tool_events(run_result.get("messages"))
    except ValueError:
        return fail("invalid_execution_trace")

    directories = [e for e in events if e["name"] == "list_directory"
                   and _relative_path(e["arguments"].get("relative_path")) ==
                   ↪  expected_directory]
    if not directories:
        return fail("missing_directory_evidence")
    reads = [e for e in events if e["name"] == "read_text_file"]
    if not reads:
        return fail("missing_read_evidence")
    before_read = [e for e in directories if e["message_index"] <
    ↪  reads[0]["message_index"]]
    if not before_read:
        return fail("read_before_directory")

    observation = before_read[-1]["result"]
    if observation.get("status") == "success":
        entries = observation.get("entries")
        if (not isinstance(entries, list) or not
        ↪  isinstance(observation.get("truncated"), bool)
                or not all(isinstance(e, dict) and all(isinstance(e.get(k), str)
                           for k in ("name", "type", "relative_path")) for e in
                           ↪  entries)):
            return fail("invalid_directory_evidence")
    selection = select_unique_text_file(observation)
    if selection["status"] != "success":
```

```
        return fail("directory_selection_failed",
        ↪  selection_error=selection.get("error_type"))
    target = _relative_path(selection["relative_path"])
    if target is None or PurePosixPath(target).parent.as_posix() !=
    ↪  expected_directory:
        return fail("invalid_directory_evidence")

    for event in reads:
        actual = _relative_path(event["arguments"].get("relative_path"))
        if actual != target:
            return fail("wrong_file_read", expected_path=target,
            ↪  actual_path=actual)
        if event["result"].get("status") != "success":
            return fail("read_failed",
            ↪  tool_error=event["result"].get("error_type"))
        if not isinstance(event["result"].get("content"), str):
            return fail("missing_file_content")
    return {"status": "passed", "reason": "verified_read", "relative_path": target}
```

18.4 experiment_agent/tools/script_tools.py

```
""" 只执行固定名单中的可信教学脚本；这不是任意代码执行沙箱。"""

import math
from pathlib import Path
import subprocess
import sys
from time import perf_counter


ALLOWED_SCRIPTS = {
    "normal": "labs/fixtures/process_scripts/normal.py",
    "error": "labs/fixtures/process_scripts/error.py",
    "timeout": "labs/fixtures/process_scripts/timeout.py",
}
DEFAULT_TIMEOUT_SECONDS = 2.0


def _output_text(value):
    """TimeoutExpired 的捕获输出可能仍是 bytes，统一转成可保存的文本。"""
    if value is None:
        return ""
```

```
    if isinstance(value, bytes):
        return value.decode("utf-8", errors="replace").replace("\r\n", "\n")
    return value


def run_python_script(project_root: str, script_id: str, *,
↪   timeout_seconds=DEFAULT_TIMEOUT_SECONDS) -> dict:
    """ 超时参数由可信 Python 调用方控制，不属于模型可提交的工具参数。"""
    if (isinstance(timeout_seconds, bool) or not isinstance(timeout_seconds, (int,
    ↪   float))
            or not math.isfinite(timeout_seconds) or timeout_seconds <= 0):
        raise ValueError("timeout_seconds must be a finite positive number")
    started = perf_counter()

    def finish(status, *, exit_code=None, stdout="", stderr="", timed_out=False,
    ↪   **details):
        return {"status": status, "script_id": script_id, "exit_code": exit_code,
                "stdout": _output_text(stdout), "stderr": _output_text(stderr),
                "timed_out": timed_out, "duration_seconds": perf_counter() -
                ↪   started,
                **details}

    if not isinstance(script_id, str) or script_id not in ALLOWED_SCRIPTS:
        return finish("failed", error_type="script_not_allowed")
    root = Path(project_root).resolve()
    script = (root / ALLOWED_SCRIPTS[script_id]).resolve()
    if not script.is_relative_to(root):
        return finish("failed", error_type="path_outside_project")
    if not script.exists():
        return finish("failed", error_type="file_not_found")
    if not script.is_file():
        return finish("failed", error_type="script_not_file")

    command = [sys.executable, "-I", "-X", "utf8", "-u", str(script)]
    try:
        completed = subprocess.run(
            command, cwd=root, shell=False, stdin=subprocess.DEVNULL,
            capture_output=True, text=True, encoding="utf-8", errors="replace",
            timeout=timeout_seconds, check=False,
        )
    except subprocess.TimeoutExpired as error:
        return finish("failed", error_type="tool_timeout",
        ↪   exception_type="TimeoutExpired",
                      timed_out=True, stdout=error.stdout, stderr=error.stderr)
```

```
    except OSError as error:
        return finish("failed", error_type="process_start_failed",
                      exception_type=type(error).__name__, message=str(error))

    if completed.returncode != 0:
        return finish("failed", error_type="process_failed",
        ↪  exit_code=completed.returncode,
                      stdout=completed.stdout, stderr=completed.stderr)
    return finish("success", exit_code=completed.returncode,
                  stdout=completed.stdout, stderr=completed.stderr)
```

18.5 experiment_agent/tool_policy.py

```
""" 文件读取与受控脚本执行任务的工具结果策略。"""


def decide_tool_result(result, corrections_used, max_corrections=1):
    """ 返回 continue、correct 或 stop；修正预算按整个任务累计。"""
    if not isinstance(result, dict) or result.get("status") not in {"success",
    ↪  "failed"}:
        return {"action": "stop", "reason": "invalid_tool_result"}
    if result["status"] == "success":
        return {"action": "continue", "reason": "tool_succeeded"}

    error_type = result.get("error_type")
    if error_type == "invalid_arguments":
        if corrections_used < max_corrections:
            return {"action": "correct", "reason": "ask_model_to_fix_arguments"}
        return {"action": "stop", "reason": "correction_limit_exceeded"}

    reasons = {
        "file_not_found": "required_file_missing",
        "path_outside_project": "permission_boundary",
        "unknown_tool": "tool_unavailable",
        "permission_denied": "file_access_denied",
        "is_a_directory": "expected_file",
        "not_a_directory": "invalid_parent_directory",
        "invalid_encoding": "utf8_required",
        "tool_timeout": "tool_timeout_needs_review",
        "io_error": "io_failure",
        "script_not_allowed": "script_not_authorized",
        "script_not_file": "expected_script_file",
```

```
        "process_failed": "script_process_failed",
        "process_start_failed": "script_start_failed",
        "execution_limit_exceeded": "script_execution_budget_exceeded",
    }
    return {"action": "stop", "reason": reasons.get(error_type,
    ↪  "unclassified_tool_failure")}
```

18.6 labs/l18_real_script_agent.py

```
""" 真实模型选择固定脚本；Python 根据执行观察决定继续或停止。"""

import argparse
import json
from pathlib import Path
from time import perf_counter
from urllib.request import Request, urlopen

from experiment_agent.ollama_loop import run_ollama_loop
from experiment_agent.tool_policy import decide_tool_result
from experiment_agent.tools.registry import build_tool_registry


TASKS = {
    "normal": "请实际运行一次正常计算演示脚本，根据工具返回的退出码和输出，
    ↪  说明是否执行成功。不要仅口头声称已运行。",
    "error": "请实际运行一次会触发演示异常的脚本，检查真实退出码和错误输出；失败后停止，
    ↪  不要换脚本或再次执行。",
    "timeout": "请实际运行一次长时间等待的演示脚本；超过工具时限后停止，不要重试，
    ↪  也不要改成其他脚本。",
}
EXPECTED_SCRIPT = {"normal": "normal", "error": "error", "timeout": "timeout"}


def expected_behavior_matched(case, result):
    """ 运行后核对实际脚本和控制流；不替模型选择参数，不评价回答语义。"""
    observations = [json.loads(m["content"]) for m in result["messages"]
                    if m["role"] == "tool" and m["tool_name"] ==
                    ↪  "run_python_script"]
    executions = [o for o in observations if "exit_code" in o]
    if len(executions) != 1 or executions[0].get("script_id") !=
    ↪  EXPECTED_SCRIPT[case]:
        return False
```

```
    execution = executions[0]
    if case == "normal":
        return (result["status"] == "completed" and execution["status"] ==
        ↪  "success"
                and execution["exit_code"] == 0 and not execution["timed_out"])
    error_type = "process_failed" if case == "error" else "tool_timeout"
    reason = "script_process_failed" if case == "error" else
    ↪  "tool_timeout_needs_review"
    stopped = (result["status"] == "failed" and result.get("error_type") ==
    ↪  "tool_policy_stopped"
               and result.get("tool_error_type") == error_type and
               ↪  result.get("stop_reason") == reason
               and execution["status"] == "failed" and execution.get("error_type")
               ↪  == error_type)
    if case == "error":
        return stopped and execution["exit_code"] not in (None, 0) and not
        ↪  execution["timed_out"]
    return stopped and execution["timed_out"] and execution["exit_code"] is None


def run_case(root, case, descriptions, max_steps=4):
    requests = []
    seen_messages = 0
    execution_calls = 0
    registered = build_tool_registry(str(root), enable_python_scripts=True)

    def run_once(script_id):
        nonlocal execution_calls
        if execution_calls >= 1:
            return {"status": "failed", "error_type": "execution_limit_exceeded"}
        execution_calls += 1
        return registered["run_python_script"](script_id=script_id)

    tools = {"run_python_script": run_once}

    def show_new_observations(messages):
        nonlocal seen_messages
        for message in messages[seen_messages:]:
            if message["role"] == "tool":
                print(f"Tool result [{message['tool_name']}]:", flush=True)
                print(json.dumps(json.loads(message["content"]),
                ↪  ensure_ascii=False, indent=2), flush=True)
        seen_messages = len(messages)
```

```
def chat(messages, tool_descriptions):
    show_new_observations(messages)
    payload = {
        "model": "qwen3.5:9b-q4_K_M", "messages": messages, "tools":
        ↪  tool_descriptions,
        "stream": False, "think": False, "keep_alive": "10m",
        "options": {"temperature": 0, "num_predict": 512},
    }
    requests.append(json.loads(json.dumps(payload, ensure_ascii=False)))
    print(f"\nModel request {len(requests)}/{max_steps} (HTTP timeout:
    ↪  180s)...", flush=True)
    request = Request("http://127.0.0.1:11434/api/chat",
                      data=json.dumps(payload,
                      ↪  ensure_ascii=False).encode("utf-8"),
                      headers={"Content-Type": "application/json"},
                      ↪  method="POST")
    with urlopen(request, timeout=180) as response:
        data = json.load(response)
    print("Model response:", flush=True)
    print(json.dumps(data, ensure_ascii=False, indent=2), flush=True)
    return data

print(f"\nCase: {case}\nTask: {TASKS[case]}", flush=True)
started = perf_counter()
result = run_ollama_loop(TASKS[case], chat, tools, descriptions,
                         max_steps=max_steps, tool_policy=decide_tool_result)
elapsed = perf_counter() - started
show_new_observations(result["messages"])
matched = expected_behavior_matched(case, result)
print(f"\nLoop status: {result['status']}")
print(f"Model decisions: {result['steps']}")
print(f"Execution function calls: {execution_calls}")
for decision in result["tool_decisions"]:
    print(f"Policy: {decision['action']} / {decision['reason']}")
if "answer" in result:
    print(f"Final model reply: {result['answer']}")
else:
    print("No final model reply: loop stopped before requesting another
    ↪  reply.")
if result["status"] == "failed":
    print(f"Stopped because: {result.get('tool_error_type',
    ↪  result.get('error_type'))} / {result.get('stop_reason')}")
print(f"Expected behavior matched: {matched}")
print(f"Elapsed seconds: {elapsed:.3f}")
```

```
    path = root / "reports" / f"l18_real_script_agent_{case}.json"
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps({
        "task": TASKS[case], "requests": requests, "result": result,
        "execution_function_calls": execution_calls, "expected_behavior_matched":
        ↪ matched,
        "elapsed_seconds": elapsed,
    }, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
    print(f"Saved report: {path}")
    return matched


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--case", choices=[*TASKS, "all"], default="normal")
    parser.add_argument("--max-steps", type=int, default=4)
    args = parser.parse_args()
    if args.max_steps < 1:
        parser.error("--max-steps must be at least 1")
    root = Path(__file__).resolve().parents[1]
    schema = json.loads((root / "experiment_agent/tools/run_python_script.schema.j」
    ↪ son").read_text(encoding="utf-8"))
    descriptions = [{"type": "function", "function": {
        "name": "run_python_script",
        "description": "实际运行一个固定允许的教学脚本。normal 正常计算; error
        ↪ 触发演示异常; timeout 长时间等待。每任务最多调用一次执行函数, 脚本超时2秒;
        ↪ 结果含退出码、stdout、stderr。",
        "parameters": schema,
    }}]
    print("Real local model; only run_python_script is exposed; one execution call
    ↪ per task.", flush=True)
    accepted = True
    for case in TASKS if args.case == "all" else [args.case]:
        if not run_case(root, case, descriptions, args.max_steps):
            accepted = False
    if not accepted:
        raise SystemExit("A case did not match its expected execution and stopping
        ↪ behavior. Inspect its report.")


if __name__ == "__main__":
    main()
```

18.7 exercises/g1_agent_loop.py

```
from experiment_agent.tool_policy import decide_tool_result
from experiment_agent.tools.registry import call_tool


def run_agent(task, model, tools, tool_descriptions, max_steps=5):
    """ 返回包含 status、steps、messages 的字典；详细分支见本课讲义。

    model(messages, tool_descriptions) 每次返回一个动作：
    {"type": "tool_call", "name": " 工具名", "arguments": {...}}
    或 {"type": "final", "content": " 回答"}。

    消息使用教学格式：assistant.content 是动作字典，tool.content 是结果字典。
    这不是 Ollama HTTP 原生消息格式。
    """
    if max_steps < 1:
        raise ValueError("max_steps must be at least 1")

    # 初始化
    messages = [{"role": "user", "content": task}]
    corrections_used = 0

    for step in range(1, max_steps + 1):
        # 请求并保存动作
        action = model(messages, tool_descriptions)
        messages.append({
            "role": "assistant",
            "content": action,
        })

        # 先检查外层类型，避免对 None、列表等调用 get。
        if not isinstance(action, dict):
            return {
                "status": "failed",
                "error_type": "invalid_model_response",
                "steps": step,
                "messages": messages,
            }

        # and 从左到右短路：确认正文是字符串后，才调用 strip。
        if (
            action.get("type") == "final"
```

```
        and isinstance(action.get("content"), str)
        and action["content"].strip()
    ):
        return {
            "status": "completed",
            "answer": action["content"],
            "steps": step,
            "messages": messages,
        }

    # 校验请求结构；具体参数是否合法，仍交给 call_tool 判断。
    valid_tool_call = (
        action.get("type") == "tool_call"
        and isinstance(action.get("name"), str)
        and bool(action["name"].strip())
        and "arguments" in action
    )
    if not valid_tool_call:
        return {
            "status": "failed",
            "error_type": "invalid_model_response",
            "steps": step,
            "messages": messages,
        }

    name = action["name"]
    arguments = action["arguments"]

    # 执行并保存工具结果
    result = call_tool(tools, name, arguments)
    messages.append({
        "role": "tool",
        "name": name,
        "content": result,
    })

    # 策略判断：继续、修正或停止
    decision = decide_tool_result(result, corrections_used)
    if decision["action"] == "correct":
        corrections_used += 1
        continue
    elif decision["action"] == "continue":
        continue
    else:
```

```
            return {
                "status": "failed",
                "error_type": "tool_policy_stopped",
                "tool_error_type": result.get("error_type"),
                "stop_reason": decision["reason"],
                "steps": step,
                "messages": messages,
            }

    # 循环外：返回 max_steps_exceeded
    return {
        "status": "failed",
        "error_type": "max_steps_exceeded",
        "steps": max_steps,
        "messages": messages,
    }
```

18.8 exercises/verify_metrics.py

```
import math


def verify_metrics(metrics, max_mse=0.1):
    """ 检查 mse 是否为有限、非负数值，并满足可信调用方给定的阈值。

    max_mse 由可信 Python 调用方给定，本题假定它是有限非负数值。
    bool、数值字符串、NaN、正负无穷都不是有效的 mse。
    """
    if not isinstance(metrics, dict) or "mse" not in metrics:
        return {
            "status": "failed",
            "reason": "invalid_metrics",
        }

    mse = metrics["mse"]

    # 1. 检查类型：必须是 int 或 float，且不能是 bool（因为 bool 是 int 的子类）
    if isinstance(mse, bool) or not isinstance(mse, (int, float)):
        return {
            "status": "failed",
            "reason": "invalid_metrics",
        }
```

```
    # 2. 检查数值的有效性：必须是有限数（排除 NaN、正负无穷）且非负
    if not math.isfinite(mse) or mse < 0:
        return {
            "status": "failed",
            "reason": "invalid_metrics",
        }

    # 3. 阈值判断：mse 大于 max_mse 判为不通过
    if mse > max_mse:
        return {
            "status": "failed",
            "reason": "mse_above_threshold",
            "mse": mse,
            "max_mse": max_mse,
        }

    # 4. 合法且小于等于阈值，返回通过
    return {
        "status": "passed",
        "reason": "verified_metrics",
        "mse": mse,
    }
```

18.9 experiment_agent/metrics_workflow.py

```
""" 读取项目内指标文件，解析 JSON，再复用学习者的指标验收函数。"""

import json

from exercises.verify_metrics import verify_metrics
from experiment_agent.tools.registry import call_tool


def run_metrics_file_workflow(tools, relative_path, max_mse=0.1, *,
↪   expected_run_id=None):
    """ 固定流程，无模型请求；max_mse 由可信调用方给定。

    返回 status=passed/failed、最后阶段的 result，以及各阶段 trace。
    expected_run_id 可选，由外层运行器提供；只核对标记，不证明实际训练计算。
    """
```

```
    read_result = call_tool(tools, "read_text_file", {"relative_path":
    ↪  relative_path})
    trace = [{"step": "read_text_file", "result": read_result}]
    if read_result["status"] != "success":
        return {"status": "failed", "result": read_result, "trace": trace}

    try:
        metrics = json.loads(read_result["content"])
    except json.JSONDecodeError as error:
        parse_result = {
            "status": "failed",
            "error_type": "invalid_json",
            "message": error.msg,
            "line": error.lineno,
            "column": error.colno,
        }
        trace.append({"step": "parse_json", "result": parse_result})
        return {"status": "failed", "result": parse_result, "trace": trace}

    trace.append({"step": "parse_json", "result": {"status": "success", "data":
    ↪  metrics}})
    if expected_run_id is not None:
        actual_run_id = metrics.get("run_id") if isinstance(metrics, dict) else
        ↪  None
        identity = {
            "status": "passed" if actual_run_id == expected_run_id else "failed",
            "reason": "run_id_matched" if actual_run_id == expected_run_id else
            ↪  "run_id_mismatch",
            "expected_run_id": expected_run_id,
            "actual_run_id": actual_run_id,
        }
        trace.append({"step": "check_run_id", "result": identity})
        if identity["status"] != "passed":
            return {"status": "failed", "result": identity, "trace": trace}

    verification = verify_metrics(metrics, max_mse=max_mse)
    trace.append({"step": "verify_metrics", "result": verification})
    return {"status": verification["status"], "result": verification, "trace":
    ↪  trace}
```

18.10 experiment_agent/experiment_workflow.py

```
""" 为可信固定脚本分配新目录，执行后只验收该目录内的指标。"""

import math
from pathlib import Path
import subprocess
import sys
from time import perf_counter
from uuid import uuid4

from experiment_agent.metrics_workflow import run_metrics_file_workflow
from experiment_agent.tools.registry import build_tool_registry
from experiment_agent.tools.script_tools import _output_text


EXPERIMENT_SCRIPT = "labs/fixtures/experiment_scripts/write_metrics.py"
EXPERIMENT_CASES = ("normal", "missing_metrics", "wrong_run_id", "error",
↪  "timeout")
TRAINING_SCRIPT = "experiments/linear_regression/train.py"
TRAINING_CASES = ("linear_regression", "linear_regression_slow")


def _execute(script, run_directory, run_id, case, timeout_seconds):
    """ 仅供可信工作流调用，脚本固定，参数来自本程序，不经 shell。"""
    started = perf_counter()

    def finish(status, *, exit_code=None, stdout="", stderr="", timed_out=False,
    ↪  **details):
        return {
            "status": status, "run_id": run_id, "case": case,
            "exit_code": exit_code, "stdout": _output_text(stdout),
            "stderr": _output_text(stderr), "timed_out": timed_out,
            "duration_seconds": perf_counter() - started, **details,
        }

    command = [
        sys.executable, "-I", "-X", "utf8", "-u", str(script),
        "--run-id", run_id, "--case", case,
    ]
    try:
        completed = subprocess.run(
            command, cwd=run_directory, shell=False, stdin=subprocess.DEVNULL,
            capture_output=True, text=True, encoding="utf-8", errors="replace",
```

```
            timeout=timeout_seconds, check=False,
        )
    except subprocess.TimeoutExpired as error:
        return finish(
            "failed", error_type="tool_timeout", exception_type="TimeoutExpired",
            timed_out=True, stdout=error.stdout, stderr=error.stderr,
        )
    except OSError as error:
        return finish("failed", error_type="process_start_failed",
        ↪  message=str(error))

    if completed.returncode != 0:
        return finish(
            "failed", error_type="process_failed", exit_code=completed.returncode,
            stdout=completed.stdout, stderr=completed.stderr,
        )
    return finish("success", exit_code=0, stdout=completed.stdout,
    ↪  stderr=completed.stderr)


def run_experiment_workflow(project_root, case="normal", *, max_mse=0.1,
↪  timeout_seconds=2.0):
    """ 固定教学 workflow；不注册为模型工具，也不接受任意脚本或输出路径。

    新目录隔离 + 成功执行 + 文件内 run_id 一致，关联本次可信教学运行。
    原故障案例使用固定指标；TRAINING_CASES 执行真实小型线性回归。
    运行编号不是防伪证明，此执行器也不是完整沙箱。
    """
    if (isinstance(timeout_seconds, bool) or not isinstance(timeout_seconds, (int,
    ↪  float))
            or not math.isfinite(timeout_seconds) or timeout_seconds <= 0):
        raise ValueError("timeout_seconds must be a finite positive number")
    if not isinstance(case, str) or case not in EXPERIMENT_CASES + TRAINING_CASES:
        return {"status": "failed", "result": {
            "status": "failed", "error_type": "script_not_allowed",
        }, "trace": []}

    root = Path(project_root).resolve()
    script_path = TRAINING_SCRIPT if case in TRAINING_CASES else EXPERIMENT_SCRIPT
    script = (root / script_path).resolve()
    if not script.is_relative_to(root):
        return {"status": "failed", "result": {
            "status": "failed", "error_type": "path_outside_project",
        }, "trace": []}
```

```
    if not script.is_file():
        return {"status": "failed", "result": {
            "status": "failed", "error_type": "file_not_found" if not
            ↪  script.exists() else "script_not_file",
        }, "trace": []}

    run_id = uuid4().hex
    relative_directory = f"runs/{run_id}"
    run_directory = (root / relative_directory).resolve()
    context = {
        "run_id": run_id, "run_directory": relative_directory,
        "metrics_path": f"{relative_directory}/metrics.json",
    }
    trace = []

    def finish(result, **details):
        return {**context, "status": result["status"], "result": result, "trace":
        ↪  trace, **details}

    if not run_directory.is_relative_to(root):
        return finish({"status": "failed", "error_type": "path_outside_project"})
    try:
        # 原子地拒绝复用已存在的叶目录；即使编号碰撞，也不覆盖旧结果。
        run_directory.mkdir(parents=True, exist_ok=False)
    except OSError as error:
        return finish({"status": "failed", "error_type": "run_setup_failed",
        ↪  "message": str(error)})
    trace.append({"step": "prepare_run", "result": {"status": "success",
    ↪  **context}})

    execution = _execute(script, run_directory, run_id, case, timeout_seconds)
    trace.append({"step": "run_python_script", "result": execution})
    if execution["status"] != "success":
        return finish(execution, execution=execution)

    # 读取权限缩小到本次目录，固定文件名，不采用 stdout 建议的其他路径。
    tools = build_tool_registry(str(run_directory))
    checked = run_metrics_file_workflow(
        tools, "metrics.json", max_mse=max_mse, expected_run_id=run_id,
    )
    trace.extend(checked["trace"])
    return finish(checked["result"], execution=execution)
```

18.11 experiments/linear_regression/train.py

```
"""CPU 合成数据线性回归：固定配置、全批量梯度下降、独立验证集。"""

import argparse
import hashlib
import json
import math
from pathlib import Path
import random
import sys
from time import perf_counter


LEARNING_RATES = {"linear_regression": 0.1, "linear_regression_slow": 0.01}


def make_data(samples, seed, *, weight=3.0, bias=2.0, noise_std=0.1):
    rng = random.Random(seed)
    data = []
    for _ in range(samples):
        x = rng.uniform(-1.0, 1.0)
        y = weight * x + bias + rng.gauss(0.0, noise_std)
        data.append({"x": x, "y": y})
    return data


def mse(data, weight, bias):
    return math.fsum((weight * row["x"] + bias - row["y"]) ** 2 for row in data) /
    ↪  len(data)


def fit_line(train_data, learning_rate, steps):
    """ 只接收训练集；验证数据不参与梯度、早停或训练步数选择。"""
    weight, bias = 0.0, 0.0
    for _ in range(steps):
        errors = [weight * row["x"] + bias - row["y"] for row in train_data]
        grad_weight = 2.0 * math.fsum(error * row["x"] for error, row in
        ↪  zip(errors, train_data)) / len(train_data)
        grad_bias = 2.0 * math.fsum(errors) / len(train_data)
        weight -= learning_rate * grad_weight
        bias -= learning_rate * grad_bias
    return {"weight": weight, "bias": bias}
```

```
def write_json(path, data):
    Path(path).write_text(json.dumps(data, ensure_ascii=False, indent=2,
    ↪  allow_nan=False) + "\n", encoding="utf-8")


def data_digest(data):
    content = json.dumps(data, sort_keys=True, separators=(",", ":"),
    ↪  allow_nan=False)
    return hashlib.sha256(content.encode("utf-8")).hexdigest()


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--run-id", required=True)
    parser.add_argument("--case", choices=list(LEARNING_RATES), required=True)
    args = parser.parse_args()
    config = {
        "learning_rate": LEARNING_RATES[args.case], "steps": 100,
        "train_samples": 128, "validation_samples": 64,
        "train_seed": 20261025, "validation_seed": 20261026,
        "target_weight": 3.0, "target_bias": 2.0, "noise_std": 0.1,
        "initial_weight": 0.0, "initial_bias": 0.0,
    }
    settings = {"weight": config["target_weight"], "bias": config["target_bias"],
    ↪  "noise_std": config["noise_std"]}
    train_data = make_data(config["train_samples"], config["train_seed"],
    ↪  **settings)
    validation_data = make_data(config["validation_samples"],
    ↪  config["validation_seed"], **settings)
    initial_train_mse = mse(train_data, 0.0, 0.0)
    initial_validation_mse = mse(validation_data, 0.0, 0.0)
    train_mean = math.fsum(row["y"] for row in train_data) / len(train_data)
    baseline_validation_mse = mse(validation_data, 0.0, train_mean)

    started = perf_counter()
    model = fit_line(train_data, config["learning_rate"], config["steps"])
    training_seconds = perf_counter() - started
    train_mse = mse(train_data, model["weight"], model["bias"])
    validation_mse = mse(validation_data, model["weight"], model["bias"])
    predictions = [{
        "x": row["x"], "y_true": row["y"],
        "y_pred": model["weight"] * row["x"] + model["bias"],
```

```
    } for row in validation_data]
    data = {"train": train_data, "validation": validation_data}
    metrics = {
        "run_id": args.run_id, "case": args.case, "task":
        ↪  "synthetic_linear_regression",
        "mse": validation_mse, "validation_mse": validation_mse, "train_mse":
        ↪  train_mse,
        "initial_train_mse": initial_train_mse, "initial_validation_mse":
        ↪  initial_validation_mse,
        "baseline_validation_mse": baseline_validation_mse,
        "training_seconds": training_seconds, "model": model, "config": config,
        "data_split_id": data_digest(data),
        "script_sha256": hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
        "python_version": sys.version.split()[0],
        "artifacts": {"config": "config.json", "data": "data.json", "model":
        ↪  "model.json",
                      "validation_predictions": "validation_predictions.json"},
    }
    write_json("config.json", config)
    write_json("data.json", data)
    write_json("model.json", model)
    write_json("validation_predictions.json", predictions)
    # 最后写 metrics；所有产物仍在父进程创建的本次 cwd 中。
    write_json("metrics.json", metrics)
    print(f"Run ID: {args.run_id}")
    print(f"Learning rate: {config['learning_rate']}; full-batch steps:
    ↪  {config['steps']}")
    print(f"Initial train MSE: {initial_train_mse:.8f}; final train MSE:
    ↪  {train_mse:.8f}")
    print(f"Initial validation MSE: {initial_validation_mse:.8f}; final validation
    ↪  MSE: {validation_mse:.8f}")
    print(f"Learned weight: {model['weight']:.8f}; bias: {model['bias']:.8f}")


if __name__ == "__main__":
    main()
```

18.12 labs/l21_real_linear_regression.py

```
""" 两组固定学习率的真实 CPU 线性回归；复用本次运行产物验收，无模型请求。"""

import json
```

```
from pathlib import Path
import sys

from experiment_agent.experiment_workflow import TRAINING_CASES,
↪  run_experiment_workflow
from labs.l19_metrics_file_workflow import redact_demo_root


def main():
    root = Path(__file__).resolve().parents[1]
    print(f"Python: {sys.executable}")
    print("Real CPU gradient descent on synthetic data; standard library only; no
    ↪  model API called.")
    print("Same data seeds, initialization, steps and threshold; only learning
    ↪  rate differs.")
    reports = []
    for case in TRAINING_CASES:
        result = run_experiment_workflow(root, case, max_mse=0.1)
        execution = result.get("execution", {})
        parsed = next((event["result"]["data"] for event in result["trace"]
                       if event["step"] == "parse_json" and
                       ↪  event["result"]["status"] == "success"), None)
        identity = next((event["result"] for event in result["trace"] if
        ↪  event["step"] == "check_run_id"), None)
        record = {"case": case, **result, "training_metrics": parsed}
        print(f"\nCase: {case}")
        print(f"Run ID: {result.get('run_id')}")
        print(f"Metrics path: {result.get('metrics_path')}")
        print(f"Execution: {execution.get('status')} /
        ↪  exit_code={execution.get('exit_code')}")
        print(f"Run identity: {identity['reason'] if identity else 'not_checked'}")
        if parsed is not None:
            print(f"Learning rate: {parsed['config']['learning_rate']}; steps:
            ↪  {parsed['config']['steps']}")
            print(f"Initial train MSE: {parsed['initial_train_mse']:.8f}")
            print(f"Final train MSE: {parsed['train_mse']:.8f}")
            print(f"Initial validation MSE:
            ↪  {parsed['initial_validation_mse']:.8f}")
            print(f"Mean baseline validation MSE:
            ↪  {parsed['baseline_validation_mse']:.8f}")
            print(f"Final validation MSE (mse): {parsed['mse']:.8f}")
            print(f"Learned model: {parsed['model']}")
            print(f"Training seconds: {parsed['training_seconds']:.6f}")
        print(f"Workflow status: {result['status']}")
```

```
        print(f"Reason: {result['result'].get('reason',
        ↪  result['result'].get('error_type'))}")
        reports.append(redact_demo_root(record, root))

    same_conditions = False
    if all(record["training_metrics"] is not None for record in reports):
        first, second = [record["training_metrics"] for record in reports]
        same_conditions = (
            first["data_split_id"] == second["data_split_id"]
            and {k: v for k, v in first["config"].items() if k != "learning_rate"}
            == {k: v for k, v in second["config"].items() if k != "learning_rate"}
        )
    print(f"\nSame data and non-learning-rate configuration: {same_conditions}")
    path = root / "reports" / "l21_real_linear_regression.json"
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps({"runs": reports, "same_conditions":
    ↪  same_conditions},
                               ensure_ascii=False, indent=2, allow_nan=False) +
                               ↪  "\n", encoding="utf-8")
    print(f"Saved report: {path}")
    # 指标未达标仍是有效实验；只有执行、来源关联或对照条件出错时命令失败。
    if not same_conditions or any(
        record.get("execution", {}).get("status") != "success"
        or not any(event["step"] == "check_run_id" and event["result"]["status"]
        ↪  == "passed"
                   for event in record["trace"])
        or record["result"].get("reason") not in {"verified_metrics",
        ↪  "mse_above_threshold"}
        for record in reports
    ):
        raise SystemExit("A training execution, run identity, or comparison
        ↪  condition failed. Inspect the report.")


if __name__ == "__main__":
    main()
```

19 附录 C. 固定学习资料、证据和训练产物索引

本次固定保存 168 个文件。路径指向 `source_snapshot`；manifest.json 记录公开脱敏字节的 SHA-256，不随原项目后续变化。

19.1 docs

- docs/01-roadmap.md
- docs/02-knowledge-map.md
- docs/03-resources.md
- docs/04-labs.md
- docs/05-capstone.md
- docs/06-learning-workflow.md

19.2 exercises

- exercises/g1_agent_loop.py
- exercises/select_text_file.py
- exercises/test_g1_agent_loop.py
- exercises/test_metrics_verifier.py
- exercises/test_text_file_selection.py
- exercises/test_timeout_mapping.py
- exercises/verify_metrics.py

19.3 experiment_agent

- experiment_agent/__init__.py
- experiment_agent/experiment_workflow.py
- experiment_agent/loop.py
- experiment_agent/metrics_workflow.py
- experiment_agent/ollama_loop.py
- experiment_agent/tool_policy.py
- experiment_agent/tools/__init__.py
- experiment_agent/tools/directory_tools.py
- experiment_agent/tools/file_tools.py
- experiment_agent/tools/list_directory.schema.json

- experiment_agent/tools/read_text_file.schema.json
- experiment_agent/tools/registry.py
- experiment_agent/tools/run_python_script.schema.json
- experiment_agent/tools/script_tools.py
- experiment_agent/verification.py

19.4 experiments

- experiments/linear_regression/train.py

19.5 labs

- labs/__init__.py
- labs/fixtures/experiment_scripts/write_metrics.py
- labs/fixtures/hello.txt
- labs/fixtures/process_scripts/error.py
- labs/fixtures/process_scripts/normal.py
- labs/fixtures/process_scripts/timeout.py
- labs/l01_read_file.py
- labs/l02_path_scope.py
- labs/l03_tool_registry.py
- labs/l04_tool_arguments.py
- labs/l05_tool_schema.py
- labs/l06_mock_agent_loop.py
- labs/l07_local_model_info.py
- labs/l08_first_local_chat.py
- labs/l09_first_tool_call.py
- labs/l10_tool_result_reply.py
- labs/l11_real_agent_loop.py
- labs/l12_tool_failure_policy.py

- labs/l13_tool_exceptions.py
- labs/l14_multi_tool_agent.py
- labs/l15_directory_read_workflow.py
- labs/l16_agent_task_verifier.py
- labs/l17_controlled_python_script.py
- labs/l18_real_script_agent.py
- labs/l19_metrics_file_workflow.py
- labs/l20_run_scoped_metrics.py
- labs/l21_real_linear_regression.py

19.6 learner_evidence

- learner_evidence/learner_l18_output.txt
- learner_evidence/learner_l20_output.txt
- learner_evidence/study-dialogue.md

19.7 lessons

- lessons/01-agent-loop.md
- lessons/02-first-file-tool.md
- lessons/03-path-scope.md
- lessons/04-tool-registry.md
- lessons/05-tool-arguments.md
- lessons/06-tool-json-schema.md
- lessons/07-mock-agent-loop.md
- lessons/08-model-backend.md
- lessons/09-first-local-chat.md
- lessons/10-real-tool-call.md
- lessons/11-tool-result-reply.md
- lessons/12-real-agent-loop.md

- lessons/13-tool-failure-policy.md
- lessons/14-tool-exceptions.md
- lessons/15-timeout-mapping-exercise.md
- lessons/16-directory-tool-and-selection.md
- lessons/17-directory-observation-exercise.md
- lessons/18-directory-read-workflow.md
- lessons/19-agent-task-verifier.md
- lessons/20-real-agent-verification.md
- lessons/21-controlled-python-script.md
- lessons/22-real-script-agent.md
- lessons/23-g1-core-loop-exercise.md
- lessons/24-experiment-metrics-contract.md
- lessons/25-metrics-file-workflow.md
- lessons/26-run-scoped-metrics.md
- lessons/27-real-linear-regression.md
- lessons/week01.md

19.8 notes

- notes/2026-10-21-review.md
- notes/argument-validation-run.md
- notes/controlled-python-script-run.md
- notes/directory-read-workflow-run.md
- notes/directory-tools-run.md
- notes/file-tool-run.md
- notes/first-local-chat-run.md
- notes/first-native-tool-call.md
- notes/g1-core-loop-exercise.md
- notes/lesson01-feedback.md

- notes/linear-regression-training-run.md
- notes/local-chat-timeout.md
- notes/metrics-contract-exercise.md
- notes/metrics-file-workflow-run.md
- notes/mock-loop-run.md
- notes/path-scope-run.md
- notes/real-agent-loop-run.md
- notes/real-entry-verification-run.md
- notes/real-script-agent-run.md
- notes/run-scoped-metrics-run.md
- notes/task-verifier-run.md
- notes/timeout-mapping-exercise.md
- notes/tool-exceptions-run.md
- notes/tool-failure-policy-run.md
- notes/tool-registry-run.md
- notes/tool-result-reply-run.md
- notes/tool-schema-run.md

19.9 reports

- reports/l08_first_local_chat.json
- reports/l08_initial_success.json
- reports/l08_timeout_diagnostic.json
- reports/l09_first_tool_call.json
- reports/l10_tool_result_reply.json
- reports/l11_real_agent_loop_steps1.json
- reports/l11_real_agent_loop_steps5.json
- reports/l12_tool_failure_policy.json
- reports/l13_tool_exceptions.json

- reports/l14_multi_tool_agent_discover.json
- reports/l14_multi_tool_agent_list.json
- reports/l14_multi_tool_agent_read.json
- reports/l15_directory_read_workflow.json
- reports/l16_agent_task_verifier.json
- reports/l17_controlled_python_script.json
- reports/l18_real_script_agent_error.json
- reports/l18_real_script_agent_normal.json
- reports/l18_real_script_agent_timeout.json
- reports/l19_metrics_file_workflow.json
- reports/l20_run_scoped_metrics.json
- reports/l21_real_linear_regression.json

19.10 selected_runs

- selected_runs/6ac21a7789fb4e458d0d0dacbe7dbcbe/config.json
- selected_runs/6ac21a7789fb4e458d0d0dacbe7dbcbe/data.json
- selected_runs/6ac21a7789fb4e458d0d0dacbe7dbcbe/metrics.json
- selected_runs/6ac21a7789fb4e458d0d0dacbe7dbcbe/model.json
- selected_runs/6ac21a7789fb4e458d0d0dacbe7dbcbe/validation_predictions.json
- selected_runs/bfee53d9e20d4112ac873487694cbfad/config.json
- selected_runs/bfee53d9e20d4112ac873487694cbfad/data.json
- selected_runs/bfee53d9e20d4112ac873487694cbfad/metrics.json
- selected_runs/bfee53d9e20d4112ac873487694cbfad/model.json
- selected_runs/bfee53d9e20d4112ac873487694cbfad/validation_predictions.json

19.11 templates

- templates/experiment-report.md
- templates/study-note.md

19.12 tests

- tests/test_directory_read_workflow.py
- tests/test_directory_tools.py
- tests/test_experiment_workflow.py
- tests/test_file_tools.py
- tests/test_linear_regression_training.py
- tests/test_loop.py
- tests/test_metrics_file_workflow.py
- tests/test_ollama_loop.py
- tests/test_real_entry_verification.py
- tests/test_real_script_agent.py
- tests/test_script_tools.py
- tests/test_task_verifier.py
- tests/test_tool_exceptions.py
- tests/test_tool_failure_policy.py
- tests/test_tool_registry.py

19.13 项目入口与环境

- .gitignore
- PROGRESS.md
- README.md
- SECURITY.md
- environment.yml
- gitattributes.snapshot.txt

20 附录 D. 学习者对话与终端证据摘录

对话摘录由本次会话整理，不伪装成完整原始聊天导出。原始终端附件另列在资料索引；私人附件位置已移除。

20.1 对话摘录的来源说明

本文件根据当前教学会话摘录关键学习问题、回答及核对结果，不是完整聊天原始导出。没有终端附件的课程，以学习者提供的会话输出和 notes 为证，不补造原始文件。

20.2 目录选择、读取和证据

学习者在第 17 课初次提交后，自行将数量判断和最后 return 移到 for 外，复验 8 项通过。

学习者解释：“因为有多个文本文件，selection 返回 failed，进入 if selection[”status”] != ”success” 分支后，return 就结束整个函数了，所以不会执行后面的 `read_text_file`。”核对：正确。

学习者解释：“是 `read_text_file` 读取时返回了 `invalid_encoding`，最外层的 status 取自 `read_result`[”status”]，所以选中文件成功不代表读取成功。”核对：正确；异常具体由底层抛出、分发器转换。

学习者指出错读证据：`expected_path`=data/hello.txt，`actual_path`=other/other.txt。核对：不能因模型声称已完成而通过。

学习者区分：“前者说明循环结束，后者说明实际读取证据通过验收。”对应真实 l14 completed + passed，4.993 秒。

20.3 脚本与消息

学习者解释 normal：“stderr 也可以输出诊断信息，不能只看它是否为空；`exit_code` 是 0，而且没有超时，所以 status 是 success。”核对：正确。

学习者解释 error：“Python 保存工具失败结果后，由策略直接停止，没有再次调用模型，所以模型还没看到这次失败结果。”核对：正确。

20.4 核心循环的实现过程

学习者索取初始化、模型调用和后续四步提示，随后尝试实现。初次读到未保存骨架；之后读到条件未连接、缩进脱离函数和循环的代码，continue 导致语法错误。学习者明确请求 AI 修复，AI 完成修复并通过 8 项测试。代码实现能力因此不能记为无提示独立完成。

学习者发现打开的文件仍显示旧版；核对时同路径磁盘内容确有变化，曾由修正版变回骨架。AI 重新写入并验证测试前后源码哈希一致，用户随后确认看到修正版。未确认覆盖发生的具体编辑器或同步来源。

20.5 指标与 JSON

学习者判断："应该拒绝，因为 \"0.04\" 是字符串，不是真正的数值。" 核对：正确。前两个字段检查获得提示后，学习者提交完整 `verify_metrics`；AI 检查并验收 8 项通过，未改写其提交。

学习者解释："读取成功只代表拿到了文件正文，json.loads 解析时抛出 JSONDecodeError，进入 except 分支后 return 就结束整个函数了，所以不会执行后面的 `verify_metrics`。" 核对：正确；l19 五例、8 项测试通过，0.028 秒。

20.6 本次产物与真实训练

学习者先问："进程成功、旧文件也存在。是不是因为 Workflow status: failed，所以找不到指标?" 核对：因果顺序应反过来，本次目录缺文件导致失败。学习者随后正确复述："是本次目录里没有生成指标文件，才导致流程失败。" l20 五例与 12 项测试通过，0.842 秒。

学习者解释真实训练第二组："虽然进程正常退出、`run_id` 也匹配，但验证集 MSE 约为 0.87745，超过了 0.1 的阈值，所以是指标验收失败，不是 Python 执行失败。" 核对：正确；l21 两组真实训练与指定 7 项测试通过，0.528 秒。

学习者最终请求按第一次标准归档全部课程细节，提供 PDF、LaTeX、Markdown 并上传 GitHub。本文件和讲义不将这次归档视为新课程、训练配置决策或 G1/G2 正式验收。